LAPORAN TUGAS BESAR 1

IF3070 DASAR INTELIGENSI ARTIFISIAL

Implementasi Algoritma Local Search untuk Optimasi Penempatan Kendaraan pada Geladak Kapal Kargo



Disusun oleh:

Kelompok 34 (TIRTA)

Mishael Gilland 18224005

Bryant Azraqi Mohammad 18224067

Stephanie Mae 18224117

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

BAB I DESKRIPSI PERSOALAN....2
1.1 Latar Belakang....2
1.2 Variabel Permasalahan....2
1.3 Objective Function dan Constraint....2
1.4 Ketentuan Representasi....2
BAB II PEMBAHASAN....4
2.1 Rancangan Solusi....4
2.1.1 Representasi State....4
2.1.2 Pembentukan State Awal....4
2.1.3 Objective Function....5
2.1.4 Neighbor dan Move....5
2.1.5 Keputusan Implementasi....5
2.2 Implementasi Algoritma Local Search....6
2.2.1 Struktur Program....6
2.2.2 Hill-Climbing....6
2.2.3 Simulated Annealing....9
2.2.4 Genetic Algorithm....13
2.2.5 Pencatatan dan Visualisasi....17
2.3 Hasil Eksperimen....21
2.3.1 Konfigurasi Eksperimen....21
2.3.2 Eksperimen Hill-Climbing....22
2.3.3 Eksperimen Simulated Annealing....46
2.3.4 Eksperimen Genetic Algorithm....54
2.4 Analisis Hasil Eksperimen....82
2.4.1 Kedekatan dengan Global Optima....82
2.4.2 Perbandingan Hasil Antar Algoritma....82
2.4.3 Perbandingan Durasi Pencarian....83
2.4.4 Konsistensi Hasil....83
2.4.5 Pengaruh Iterasi dan Populasi pada Genetic Algorithm....84
BAB III KESIMPULAN DAN SARAN....85
3.1 Kesimpulan....85
3.2 Saran....85
PEMBAGIAN TUGAS....86
REFERENSI....87
LAMPIRAN....88
Lampiran A. Form Penggunaan Generative AI....89
Lampiran B. Tautan Repository....89

DAFTAR TABEL

Tabel 1. Daftar variabel permasalahan.....4
Tabel 2. Atribut penempatan kendaraan dalam state.....6
Tabel 3. Jenis move dan perubahan state.....7
Tabel 4. Modul program dan perannya.....8
Tabel 5. Fungsi Hill-Climbing.....9
Tabel 6. Fungsi Simulated Annealing.....12
Tabel 7. Fungsi Genetic Algorithm.....15
Tabel 8. Berkas keluaran eksperimen.....20
Tabel 9. Fungsi visualisasi.....20
Tabel 10. Konfigurasi parameter eksperimen.....23
Tabel 11. Hasil tiga pengulangan stochastic Hill-Climbing.....24
Tabel 12. [BONUS] Hasil eksperimen varian Hill-Climbing.....29
Tabel 13. [BONUS] Jumlah langkah sideways tiap pengulangan.....29
Tabel 14. [BONUS] Jumlah restart dan iterasi tiap pencarian.....30
Tabel 15. Hasil tiga pengulangan Simulated Annealing.....46
Tabel 16. Hasil GA dengan populasi 10 dan variasi jumlah iterasi.....52
Tabel 17. Hasil GA dengan 30 iterasi dan variasi jumlah populasi.....66
Tabel 18. Perbandingan hasil dan durasi seluruh konfigurasi.....80
Tabel 19. Pembagian tugas anggota kelompok.....84
Tabel 20. Deklarasi penggunaan AI.....87

DAFTAR GAMBAR

Gambar 1. State awal stochastic Hill-Climbing (run 1, seed 40)....27
Gambar 2. State akhir stochastic Hill-Climbing (run 1, seed 40)....27
Gambar 3. Kurva objektif stochastic Hill-Climbing (run 1, seed 40)....28
Gambar 4. State awal stochastic Hill-Climbing (run 2, seed 41)....28
Gambar 5. State akhir stochastic Hill-Climbing (run 2, seed 41)....29
Gambar 6. Kurva objektif stochastic Hill-Climbing (run 2, seed 41)....29
Gambar 7. State awal stochastic Hill-Climbing (run 3, seed 42)....30
Gambar 8. State akhir stochastic Hill-Climbing (run 3, seed 42)....30
Gambar 9. Kurva objektif stochastic Hill-Climbing (run 3, seed 42)....31
Gambar 10. State awal steepest ascent (run 1, seed 40)....32
Gambar 11. State akhir steepest ascent (run 1, seed 40)....33
Gambar 12. Kurva objektif steepest ascent (run 1, seed 40)....33
Gambar 13. State awal steepest ascent (run 2, seed 41)....34
Gambar 14. State akhir steepest ascent (run 2, seed 41)....35
Gambar 15. Kurva objektif steepest ascent (run 2, seed 41)....35
Gambar 16. State awal steepest ascent (run 3, seed 42)....36
Gambar 17. State akhir steepest ascent (run 3, seed 42)....37
Gambar 18. Kurva objektif steepest ascent (run 3, seed 42)....37
Gambar 19. State awal sideways move (run 1, seed 40)....38
Gambar 20. State akhir sideways move (run 1, seed 40)....39
Gambar 21. Kurva objektif sideways move (run 1, seed 40)....39
Gambar 22. State awal sideways move (run 2, seed 41)....40
Gambar 23. State akhir sideways move (run 2, seed 41)....40
Gambar 24. Kurva objektif sideways move (run 2, seed 41)....41
Gambar 25. State awal sideways move (run 3, seed 42)....41
Gambar 26. Kurva objektif sideways move (run 3, seed 42)....42
Gambar 27. State akhir sideways move (run 3, seed 42)....42
Gambar 28. State awal random restart (run 1, seed 40)....43
Gambar 29. State terbaik random restart (run 1, seed 40)....44
Gambar 30. Kurva objektif random restart (run 1, seed 40)....44
Gambar 31. State awal random restart (run 2, seed 41)....45
Gambar 32. State terbaik random restart (run 2, seed 41)....45
Gambar 33. Kurva objektif random restart (run 2, seed 41)....46
Gambar 34. State awal random restart (run 3, seed 42)....46
Gambar 35. State terbaik random restart (run 3, seed 42)....47
Gambar 36. Kurva objektif random restart (run 3, seed 42)....47
Gambar 37. State awal Simulated Annealing (run 1, seed 40)....48
Gambar 38. State akhir terbaik Simulated Annealing (run 1, seed 40)....49
Gambar 39. Kurva objektif Simulated Annealing (run 1, seed 40)....49
Gambar 40. Kurva e^(ΔE/T) Simulated Annealing (run 1, seed 40)....49

Gambar 41. State awal Simulated Annealing (run 2, seed 41).....50
Gambar 42. State akhir terbaik Simulated Annealing (run 2, seed 41).....50
Gambar 43. Kurva objektif Simulated Annealing (run 2, seed 41).....51
Gambar 44. Kurva e^(ΔE/T) Simulated Annealing (run 2, seed 41)..... 51
Gambar 45. State awal Simulated Annealing (run 3, seed 42).....51
Gambar 46. State akhir terbaik Simulated Annealing (run 3, seed 42).....52
Gambar 47. Kurva objektif Simulated Annealing (run 3, seed 42).....52
Gambar 48. Kurva e^(ΔE/T) Simulated Annealing (run 3, seed 42)..... 52
Gambar 49. State awal terbaik GA (populasi 10, 1 iterasi, run 1, seed 40)..... 55
Gambar 50. State akhir terbaik GA (populasi 10, 1 iterasi, run 1, seed 40).....55
Gambar 51. Kurva objektif maksimum dan rata-rata GA (populasi 10, 1 iterasi, run 1, seed 40).....56
Gambar 52. State awal terbaik GA (populasi 10, 1 iterasi, run 2, seed 41)..... 56
Gambar 53. State akhir terbaik GA (populasi 10, 1 iterasi, run 2, seed 41).....57
Gambar 54. Kurva objektif maksimum dan rata-rata GA (populasi 10, 1 iterasi, run 2, seed 41).....57
Gambar 55. State awal terbaik GA (populasi 10, 1 iterasi, run 3, seed 42)..... 58
Gambar 56. State akhir terbaik GA (populasi 10, 1 iterasi, run 3, seed 42).....58
Gambar 57. Kurva objektif maksimum dan rata-rata GA (populasi 10, 1 iterasi, run 3, seed 42).....59
Gambar 58. State awal terbaik GA (populasi 10, 10 iterasi, run 1, seed 40)..... 59
Gambar 59. State akhir terbaik GA (populasi 10, 10 iterasi, run 1, seed 40).....60
Gambar 60. Kurva objektif maksimum dan rata-rata GA (populasi 10, 10 iterasi, run 1, seed 40).....60
Gambar 61. State awal terbaik GA (populasi 10, 10 iterasi, run 2, seed 41)..... 61
Gambar 62. State akhir terbaik GA (populasi 10, 10 iterasi, run 2, seed 41).....61
Gambar 63. Kurva objektif maksimum dan rata-rata GA (populasi 10, 10 iterasi, run 2, seed 41).....62
Gambar 64. State awal terbaik GA (populasi 10, 10 iterasi, run 3, seed 42)..... 62
Gambar 65. State akhir terbaik GA (populasi 10, 10 iterasi, run 3, seed 42).....63
Gambar 66. Kurva objektif maksimum dan rata-rata GA (populasi 10, 10 iterasi, run 3, seed 42).....63
Gambar 67. State awal terbaik GA (populasi 10, 100 iterasi, run 1, seed 40)..... 64
Gambar 68. State akhir terbaik GA (populasi 10, 100 iterasi, run 1, seed 40).....64
Gambar 69. Kurva objektif maksimum dan rata-rata GA (populasi 10, 100 iterasi, run 1, seed 40).....65
Gambar 70. State awal terbaik GA (populasi 10, 100 iterasi, run 2, seed 41)..... 65
Gambar 71. State akhir terbaik GA (populasi 10, 100 iterasi, run 2, seed 41).....66
Gambar 72. Kurva objektif maksimum dan rata-rata GA (populasi 10, 100 iterasi, run 2, seed 41).....66
Gambar 73. State awal terbaik GA (populasi 10, 100 iterasi, run 3, seed 42)..... 67
Gambar 74. State akhir terbaik GA (populasi 10, 100 iterasi, run 3, seed 42).....67

Gambar 75. Kurva objektif maksimum dan rata-rata GA (populasi 10, 100 iterasi, run 3, seed 42).....68
Gambar 76. State awal terbaik GA (populasi 2, 30 iterasi, run 1, seed 40)..... 69
Gambar 77. State akhir terbaik GA (populasi 2, 30 iterasi, run 1, seed 40).....69
Gambar 78. Kurva objektif maksimum dan rata-rata GA (populasi 2, 30 iterasi, run 1, seed 40).....70
Gambar 79. State awal terbaik GA (populasi 2, 30 iterasi, run 2, seed 41)..... 70
Gambar 80. State akhir terbaik GA (populasi 2, 30 iterasi, run 2, seed 41).....71
Gambar 81. Kurva objektif maksimum dan rata-rata GA (populasi 2, 30 iterasi, run 2, seed 41).....71
Gambar 82. State awal terbaik GA (populasi 2, 30 iterasi, run 3, seed 42)..... 72
Gambar 83. State akhir terbaik GA (populasi 2, 30 iterasi, run 3, seed 42).....72
Gambar 84. Kurva objektif maksimum dan rata-rata GA (populasi 2, 30 iterasi, run 3, seed 42).....73
Gambar 85. State awal terbaik GA (populasi 10, 30 iterasi, run 1, seed 40)..... 73
Gambar 86. State akhir terbaik GA (populasi 10, 30 iterasi, run 1, seed 40).....74
Gambar 87. Kurva objektif maksimum dan rata-rata GA (populasi 10, 30 iterasi, run 1, seed 40).....74
Gambar 88. State awal terbaik GA (populasi 10, 30 iterasi, run 2, seed 41)..... 75
Gambar 89. State akhir terbaik GA (populasi 10, 30 iterasi, run 2, seed 41).....75
Gambar 90. Kurva objektif maksimum dan rata-rata GA (populasi 10, 30 iterasi, run 2, seed 41).....76
Gambar 91. State awal terbaik GA (populasi 10, 30 iterasi, run 3, seed 42)..... 76
Gambar 92. State akhir terbaik GA (populasi 10, 30 iterasi, run 3, seed 42).....77
Gambar 93. Kurva objektif maksimum dan rata-rata GA (populasi 10, 30 iterasi, run 3, seed 42).....77
Gambar 94. State awal terbaik GA (populasi 30, 30 iterasi, run 1, seed 40)..... 78
Gambar 95. State akhir terbaik GA (populasi 30, 30 iterasi, run 1, seed 40).....78
Gambar 96. Kurva objektif maksimum dan rata-rata GA (populasi 30, 30 iterasi, run 1, seed 40).....79
Gambar 97. State awal terbaik GA (populasi 30, 30 iterasi, run 2, seed 41)..... 79
Gambar 98. State akhir terbaik GA (populasi 30, 30 iterasi, run 2, seed 41).....80
Gambar 99. Kurva objektif maksimum dan rata-rata GA (populasi 30, 30 iterasi, run 2, seed 41).....80
Gambar 100. State awal terbaik GA (populasi 30, 30 iterasi, run 3, seed 42)..... 81
Gambar 101. State akhir terbaik GA (populasi 30, 30 iterasi, run 3, seed 42).....81
Gambar 102. Kurva objektif maksimum dan rata-rata GA (populasi 30, 30 iterasi, run 3, seed 42).....82

BAB I DESKRIPSI PERSOALAN

1.1 Latar Belakang

PT Intim Line membutuhkan penataan kendaraan pada geladak kapal dengan ruang dan kapasitas terbatas. Setiap kendaraan memiliki ukuran, berat, dan tarif pengiriman yang berbeda. Program menentukan kendaraan yang dimuat beserta posisi dan orientasinya. Pencarian dilakukan menggunakan Hill-Climbing, Simulated Annealing, dan Genetic Algorithm.

1.2 Variabel Permasalahan

Tabel 1. Daftar variabel permasalahan

Entitas	Atribut	Keterangan
Kendaraan	id	Identitas kendaraan
	lebar, panjang	Ukuran dalam sel grid
	berat	Berat dalam unit
	tarif_kirim	Tarif dalam **juta rupiah**
	eta_hari	Estimated Time of Arrival (sisa hari)
Kapal	id	Identitas Kapal
	lebar, panjang	Ukuran geladak (satuan grid)
	kapasitas	Kapasitas angkut maksimum (unit)

1.3 Objective Function dan Constraint

Objektif tugas ini adalah memaksimalkan total tarif kirim kendaraan yang dimuat:

$f(s) = \Sigma$ tarif_kirim(i), untuk setiap kendaraan i yang dimuat pada state s.

Kendaraan yang berada di luar kapal tidak menambah nilai objective.

Solusi harus memenuhi batasan:

- Kendaraan tidak saling bertumpuk.
- Seluruh bagian kendaraan yang dimuat berada di dalam dek.
- Total berat muatan tidak melebihi kapasitas kapal.

Seluruh state yang tidak valid akan ditolak. Pada Algoritma Genetic, penempatan ‘anak’ yang melanggar constraint diperbaiki dengan mengembalikan kendaraan terkait ke luar kapal. Tidak digunakan penalti pada fitness.

1.4 Ketentuan Representasi

- Kendaraan dan geladak direpresentasikan sebagai grid dua dimensi berbentuk persegi atau persegi panjang dengan koordinat bilangan bulat.
- Kendaraan tidak harus seluruhnya dimuat, kendaraan boleh berada di luar kapal (seluruh bagian kendaraan di luar kapal).
- *Initial state* dibangkitkan secara acak.
- *Neighbor* dibentuk dengan salah satu langkah: menukar dua kendaraan, memindahkan kendaraan ke koordinat lain, atau mengubah orientasi kendaraan.
- *Final state* menunjukkan keadaan semua kendaraan (di dalam maupun di luar kapal), keadaan geladak, dan nilai *objective function*.

BAB II PEMBAHASAN

2.1 Rancangan Solusi

2.1.1 Representasi State

State direpresentasikan menggunakan ***dictionary***. Setiap ID kendaraan menjadi kunci yang memiliki nilai berupa penempatan atau ***None***. Nilai ***None*** menunjukkan kendaraan berada di luar kapal.

Penempatan kendaraan berisi empat atribut:

Tabel 2. Atribut penempatan kendaraan dalam state

Atribut	Isi
kapal_id	ID kapal tempat kendaraan dimuat
x	Koordinat horizontal sudut kiri atas kendaraan
y	Koordinat vertikal sudut kiri atas kendaraan
orientasi	vertikal atau horizontal

Contoh potongan state:

```
{
 "lcgc_01": {"kapal_id": "kapal_utama", "x": 0, "y": 0, "orientasi": "vertikal"},
 "lcgc_02": null
}
```

Koordinat menunjukkan sudut kiri atas kendaraan. Titik asal berada di kiri atas dek, x ke kanan, dan y ke bawah. Orientasi horizontal menukar lebar dan panjang. Pemeriksaannya terdapat pada ukuran(), _sel(), dan periksa_state() dalam data.py.

2.1.2 Pembentukan State Awal

Semua kendaraan awalnya berada di luar kapal. Program mengacak urutan kendaraan, membentuk pilihan penempatan valid, lalu memilih satu posisi atau ***None*** secara acak. Sel terisi dan berat muatan diperbarui setiap kali kendaraan dimuat.

Pengacakan memakai *random.Random(seed)* agar dapat diulang. Implementasi terdapat pada inisialisasi_acak() dalam data.py. Pada Algoritma Genetic, buat_populasi() menambahkan state acak lainnya. Individu terbaik populasi awal digunakan sebagai state awal representatif pada hasil.

2.1.3 Objective Function

Fungsi nilai_objektif() memanggil validasi sebelum menjumlahkan tarif kendaraan yang dimuat. Validasi memeriksa kelengkapan ID, koordinat, orientasi, batas dek, tabrakan, dan kapasitas.

```
def nilai_objektif(kapal, kendaraan, state):
    #Jumlah tarif kirim kendaraan yang dimuat.
    periksa_state(kapal, kendaraan, state)
    return sum(k["tarif_kirim"] for k in kendaraan if state[k["id"]] is not None)
```

Kode terdapat pada data.py. Perubahan posisi kendaraan di dalam kapal dapat mempertahankan objective, tetapi membuka ruang untuk kendaraan lain.

2.1.4 Neighbor dan Move

Neighbor dibentuk melalui pindah, putar, atau tukar. Hasil perubahan harus berbeda dari *state* awal dan memenuhi *constraint*.

Tabel 3. Jenis move dan perubahan state

Move	Perubahan
Pindah	Mengubah posisi, memasukkan, atau mengeluarkan kendaraan
Putar	Menukar orientasi pada koordinat yang sama
Tukar	Menukar penempatan dua kendaraan

Stochastic Hill-Climbing dan *Simulated Annealing* memakai tetangga_acak(), sedangkan steepest dan sideways memakai semua_tetangga(). Implementasinya terdapat pada tetangga.py. Sampling yang gagal mengembalikan None.

2.1.5 Keputusan Implementasi

Pada HC, kandidat terbaik dengan nilai seri dipilih secara acak. Pada GA, anak diperbaiki dengan memeriksa gen dalam urutan acak dan mengembalikan penempatan tidak valid ke None. Elitisme mempertahankan satu individu terbaik pada generasi berikutnya. Implementasi terdapat pada _steepest() dalam hill_climbing.py, serta perbaiki() dan cari() dalam genetic.py.

2.2 Implementasi Algoritma Local Search

Bagian ini menjelaskan kelas dan fungsi yang diimplementasikan dalam bahasa Python beserta source code-nya.

2.2.1 Struktur Program

Program membaca dataset, membentuk state awal, menjalankan pencarian, menyimpan hasil, lalu membuat gambar. Setiap algoritma menyediakan cari() yang mengembalikan state terbaik, riwayat, dan metrik.

Tabel 4. Modul program dan perannya

Modul	Peran
data.py	Data, state awal, validasi, dan *objective*
tetangga.py	*Neighbor* dan pemeriksaan *local maximum*
hill_climbing.py	seluruh varian *Hill Climbing*
simulated_annealing.py	Algoritma *Simulated Annealing*
genetic.py	Algoritma Genetic
eksperimen.py	Pengulangan dan ringkasan eksperimen
pencatatan.py	Memasukkan data eksperimen dalam bentuk JSON dan CSV
visualisasi.py	Gambar state dan grafik

2.2.2 Hill-Climbing

Varian wajib yang digunakan adalah stochastic Hill-Climbing. Algoritma mengambil satu tetangga acak dan menerimanya jika objective meningkat. Parameter maks_gagal membatasi jumlah kandidat berturut-turut yang tidak memperbaiki nilai. Pencarian berhenti ketika batas tercapai atau generator tidak menemukan kandidat valid.

[BONUS] Steepest ascent mengevaluasi seluruh tetangga valid dan memilih objective tertinggi. Pencarian berhenti ketika tidak ada peningkatan. Sideways move menggunakan

pemilihan yang sama, tetapi dapat menerima objective yang setara untuk mengubah susunan kendaraan. Parameter maks_sideways membatasi total langkah dengan nilai sama.

[BONUS] Random restart mengulang steepest ascent dari state acak baru dan menyimpan solusi terbaik seluruh pencarian. Parameter maks_restart menghitung pengulangan setelah pencarian pertama, sehingga sepuluh restart berarti sebelas pencarian. Seluruh varian diimplementasikan melalui _stochastic(), _steepest(), dan cari() dalam hill_climbing.py.

Tabel 5. Fungsi Hill-Climbing

Kelas/Fungsi	Deskripsi
_stochastic()	Mengambil tetangga acak dan menerima peningkatan objective
_steepest()	Menjalankan *steepest ascent* dan *sideways move*
cari()	Memeriksa parameter, memilih varian, menjalankan restart, dan mengembalikan hasil.

```python
import random
from copy import deepcopy

from .data import inisialisasi_acak, nilai_objektif
from .tetangga import semua_tetangga, tetangga_acak

VARIAN = ("stochastic", "steepest", "sideways", "random_restart")


def _stochastic(kapal, kendaraan, state, acak, riwayat, maks_gagal):
    nilai, gagal = riwayat[-1]["objektif"], 0
    while gagal < maks_gagal:
        hasil = tetangga_acak(kapal, kendaraan, state, acak)
        if hasil is None:
            break
        nilai_baru = nilai_objektif(kapal, kendaraan, hasil[0])
        if nilai_baru > nilai:
            state, nilai, gagal = hasil[0], nilai_baru, 0
        else:
            gagal += 1
        riwayat.append({"iterasi": len(riwayat), "objektif": nilai})
    return state
```

```
def _steepest(kapal, kendaraan, state, acak, riwayat, maks_sideways=0,
restart=None):
    """Pindah ke tetangga terbaik"""
    tarif = {k["id"]: k["tarif_kirim"] for k in kendaraan}
    nilai, sideways = riwayat[-1]["objektif"], 0
    while True:
        terbaik = max(((sum(tarif[i] for i, p in baru.items() if p is not None),
                        acak.random(), baru)
                       for baru, _ in semua_tetangga(kapal, kendaraan, state)),
                      key=lambda calon: calon[:2], default=None)
        if terbaik is None or terbaik[0] < nilai:
            break
        if terbaik[0] == nilai:
            if sideways >= maks_sideways:
                break
            sideways += 1
        nilai, state = terbaik[0], terbaik[2]
        baris = {"iterasi": len(riwayat), "objektif": nilai}
        riwayat.append(baris if restart is None else dict(baris,
restart=restart))
        if maks_sideways and sideways >= maks_sideways:
            break
    return state, sideways


def cari(kapal, kendaraan, state_awal, seed, maks_gagal=1000, *,
varian="stochastic",
         maks_sideways=100, maks_restart=10):
    if type(seed) is not int:
        raise ValueError("seed harus integer")
    for nama, nilai in (("maks_gagal", maks_gagal), ("maks_sideways",
maks_sideways),
                        ("maks_restart", maks_restart)):
        if type(nilai) is not int or nilai < 0:
            raise ValueError(f"{nama} harus integer nonnegatif")
    if varian not in VARIAN:
        raise ValueError(f"varian harus salah satu dari {VARIAN}")
    acak = random.Random(seed)
    riwayat = [{"iterasi": 0, "objektif": nilai_objektif(kapal, kendaraan,
state_awal)}]
    state = deepcopy(state_awal)
    metrik = dict(varian=varian, parameter_efektif=dict(
        varian=varian, maks_gagal=maks_gagal, maks_sideways=maks_sideways,
        maks_restart=maks_restart))
```

```
    if varian == "stochastic":
        akhir = _stochastic(kapal, kendaraan, state, acak, riwayat, maks_gagal)
        return akhir, riwayat, dict(metrik, maks_gagal=maks_gagal)
    if varian != "random_restart":
        batas = maks_sideways if varian == "sideways" else 0
        akhir, sideways = _steepest(kapal, kendaraan, state, acak, riwayat,
batas)
        return akhir, riwayat, dict(metrik, jumlah_sideways=sideways)

    riwayat[0]["restart"], iterasi_per_restart = 0, []
    terbaik = state
    for restart in range(maks_restart + 1):
        if restart:
            state = inisialisasi_acak(kapal, kendaraan, acak.randrange(2**31))
            nilai = nilai_objektif(kapal, kendaraan, state)
            riwayat.append({"iterasi": len(riwayat), "objektif": nilai,
"restart": restart})
        mulai = len(riwayat)
        state, _ = _steepest(kapal, kendaraan, state, acak, riwayat,
restart=restart)
        iterasi_per_restart.append(len(riwayat) - mulai)
        if nilai_objektif(kapal, kendaraan, state) > nilai_objektif(kapal,
kendaraan, terbaik):
            terbaik = state
    return terbaik, riwayat, dict(metrik, jumlah_restart=maks_restart,
                                  iterasi_per_restart=iterasi_per_restart)
```

2.2.3 Simulated Annealing

Simulated Annealing mengambil tetangga acak dan menerima kandidat dengan objective sama atau lebih tinggi. Kandidat yang lebih buruk dapat diterima dengan probabilitas exp((ΔE/T) × evaluation_bias), dengan ΔE sebagai selisih objective kandidat terhadap state saat ini. Penerimaan penurunan memberi kesempatan pencarian keluar dari susunan yang sulit diperbaiki secara langsung.

Suhu diturunkan melalui suhu_awal × pendinginan$^{(\text{iterasi} - 1)}$. Pencarian berhenti ketika batas iterasi tercapai atau suhu di bawah minimum. State terbaik disimpan terpisah, sehingga objective state saat ini dapat turun sementara hasil akhir tetap menyimpan solusi terbaik.

Frekuensi stuck dihitung sebagai proporsi checkpoint tanpa satu langkah valid yang langsung meningkatkan objective, termasuk plateau. Implementasi terdapat pada simulated_annealing.py, dengan pemeriksaan puncak lokal dari modul tetangga.

Tabel 6. Fungsi Simulated Annealing

Kelas/Fungsi	Deskripsi
probabilitas_penerimaan()	Menghitung peluang menerima kandidat berdasarkan selisih objektif, suhu, dan bias
_eksponen()	Menghitung nilai exp((ΔE/T) untuk grafik
_jadwal_suhu()	Menghitung suhu dan menentukan akhir dari pendinginan
cari()	Menjalankan pencarian, menyimpan *state* terbaik, dan mencatat metrik *Simulated Annealing*

```python
import math
import random
from copy import deepcopy

from .data import angka_valid, nilai_objektif
from .tetangga import puncak_lokal, tetangga_acak


def probabilitas_penerimaan(delta_e, suhu, evaluation_bias=1):
    """Bias > 1 membuat penurunan nilai lebih sulit diterima."""
    if not angka_valid(suhu) or suhu == 0:
        raise ValueError("suhu harus positif dan berhingga")
    if not angka_valid(evaluation_bias) or evaluation_bias == 0:
        raise ValueError("evaluation_bias harus positif dan berhingga")
    if type(delta_e) not in (int, float) or not math.isfinite(delta_e):
        raise ValueError("delta_e harus angka berhingga")
    return 1.0 if delta_e >= 0 else math.exp((delta_e / suhu) * evaluation_bias)


def _eksponen(delta_e, suhu):
    try:
        nilai = math.exp(delta_e / suhu)
        return nilai if math.isfinite(nilai) else None
    except OverflowError:
        return None


def _jadwal_suhu(iterasi, suhu_awal, pendinginan, suhu_minimum):
    """schedule(t) pada buku  dan suhu nol menandai akhir pencarian."""
```

```
    suhu = suhu_awal * pendinginan ** (iterasi - 1)
    return suhu if suhu >= suhu_minimum else 0


def cari(kapal, kendaraan, state_awal, seed, *,
         suhu_awal=50, pendinginan=.995, suhu_minimum=.1, batas_iterasi=1000,
         evaluation_bias=1, interval_stuck=50, tetangga=tetangga_acak,
cek_stuck=puncak_lokal):
    """Kembalikan state terbaik, riwayat state saat ini, dan metrik SA."""
    nilai = nilai_objektif(kapal, kendaraan, state_awal)
    probabilitas_penerimaan(0, suhu_awal, evaluation_bias)
    if not angka_valid(suhu_minimum) or not 0 < suhu_minimum <= suhu_awal:
        raise ValueError("suhu_minimum harus positif dan tidak melebihi
suhu_awal")
    if not angka_valid(pendinginan) or not 0 < pendinginan < 1:
        raise ValueError("pendinginan harus di antara 0 dan 1")
    if type(batas_iterasi) is not int or batas_iterasi < 0:
        raise ValueError("batas_iterasi harus integer nonnegatif")
    if type(interval_stuck) is not int or interval_stuck < 1:
        raise ValueError("interval_stuck harus integer positif")
    if type(seed) is not int:
        raise ValueError("seed harus integer")
    if not callable(tetangga):
        raise ValueError("tetangga harus berupa fungsi")
    if cek_stuck is not None and not callable(cek_stuck):
        raise ValueError("cek_stuck harus fungsi atau None (belum diukur)")
    if not angka_valid(nilai):
        raise ValueError("nilai objektif tidak berhingga")

    acak = random.Random(seed)
    state = deepcopy(state_awal)
    terbaik, nilai_terbaik = deepcopy(state), nilai
    riwayat = [dict(iterasi=0, objektif=nilai)]
    jejak, pemeriksaan = [], []
    diterima = tanpa_kandidat = eksponen_meluap = 0
    alasan = "batas_iterasi"

    for iterasi in range(1, batas_iterasi + 1):
        suhu = _jadwal_suhu(iterasi, suhu_awal, pendinginan, suhu_minimum)
        if suhu == 0:
            alasan = "suhu_minimum"
            break
        hasil = tetangga(kapal, kendaraan, deepcopy(state), acak)
        kandidat, langkah = hasil if hasil is not None else (None, None)
```

```
        eksponen = delta_e = peluang = None
        terima = False
        if kandidat is None:
            tanpa_kandidat += 1
        else:
            nilai_baru = nilai_objektif(kapal, kendaraan, kandidat)
            if not angka_valid(nilai_baru):
                raise ValueError("nilai kandidat tidak berhingga")
            delta_e = nilai_baru - nilai  # Positif berarti shipping fee membaik.
            eksponen = _eksponen(delta_e, suhu)
            eksponen_meluap += eksponen is None
            peluang = probabilitas_penerimaan(delta_e, suhu, evaluation_bias)
            terima = peluang == 1 or acak.random() < peluang
            if terima:
                state, nilai = deepcopy(kandidat), nilai_baru
                diterima += 1
                if nilai > nilai_terbaik:
                    terbaik, nilai_terbaik = deepcopy(state), nilai
        riwayat.append(dict(iterasi=iterasi, objektif=nilai,
                            eksponen_penerimaan=eksponen))
        jejak.append(dict(iterasi=iterasi, suhu=suhu, delta_e=delta_e,
langkah=deepcopy(langkah),
                          probabilitas_penerimaan=peluang, diterima=terima))
        if cek_stuck is not None and (iterasi % interval_stuck == 0
                or iterasi == batas_iterasi
                or _jadwal_suhu(iterasi + 1, suhu_awal, pendinginan,
suhu_minimum) == 0):
            stuck = cek_stuck(kapal, kendaraan, deepcopy(state))
            if type(stuck) is not bool:
                raise ValueError("cek_stuck harus mengembalikan bool")
            pemeriksaan.append(dict(iterasi=iterasi, stuck=stuck))

    jumlah_stuck = sum(p["stuck"] for p in pemeriksaan)
    definisi = "Belum diukur."
    if cek_stuck is puncak_lokal:
        definisi = "Tidak ada langkah langsung yang menaikkan objektif termasuk
plateau."
    elif cek_stuck is not None:
        definisi = "Mengikuti pemeriksa yang diberikan."
    metrik = dict(
        parameter_efektif=dict(suhu_awal=suhu_awal, pendinginan=pendinginan,
                               suhu_minimum=suhu_minimum,
batas_iterasi=batas_iterasi,
```

```
                              evaluation_bias=evaluation_bias,
interval_stuck=interval_stuck),
       alasan_berhenti=alasan, jumlah_diterima=diterima,
jumlah_tanpa_kandidat=tanpa_kandidat,
       jumlah_eksponen_overflow=eksponen_meluap,
       langkah_sa=jejak, pemeriksaan_stuck=pemeriksaan,
       jumlah_stuck=jumlah_stuck, jumlah_pemeriksaan=len(pemeriksaan),
       frekuensi_stuck=jumlah_stuck / len(pemeriksaan) if pemeriksaan else None,
       satuan_stuck="proporsi pemeriksaan", nilai_terbaik=nilai_terbaik,
       definisi_stuck=definisi,
       generator_tetangga=getattr(tetangga, "__name__",
type(tetangga).__name__),
       pemeriksa_stuck=(getattr(cek_stuck, "__name__", type(cek_stuck).__name__)
                        if cek_stuck is not None else None),
   )
   return terbaik, riwayat, metrik
```

2.2.4 Genetic Algorithm

Genetic Algorithm menggunakan satu state sebagai individu dan penempatan setiap kendaraan sebagai gen. Populasi awal dibentuk secara acak, kemudian *fitness* dihitung dari total tarif kirim. Induk dipilih menggunakan roulette wheel, sehingga individu dengan fitness lebih tinggi memiliki peluang lebih besar untuk terpilih.

Crossover satu titik menggabungkan gen dua induk menjadi dua anak. Penempatan anak yang melanggar constraint dikembalikan ke luar kapal. Mutasi memilih satu kendaraan dan mencoba penempatan valid lain untuk menambah variasi populasi.

Satu individu terbaik dipertahankan melalui elitisme. Proses diulang hingga batas_iterasi, dengan satu iterasi sebagai satu generasi pembaruan. Maksimum dan rata-rata fitness dicatat sejak generasi 0. Implementasi terdapat pada genetic.py.

Tabel 7. Fungsi Genetic Algorithm

Kelas/Fungsi	Deskripsi
buat_populasi()	Membentuk populasi dari state masukan dan state acak tambahan
pilih_induk()	Memilih dua induk berdasarkan bobot *fitness*
perbaiki()	Mengembalikan penempatan tidak valid menjadi ***None***

Kelas/Fungsi	Deskripsi
kawin_silang()	***Crossover*** : Menghasilkan dua anak dengan menyilang satu titik
mutasi()	Mengubah penempatan kendaraan berdasarkan peluang mutasi
cari()	Mengevaluasi populasi, menjalankan pergantian generasi, dan menyimpan hasil

```
import argparse
import random
from copy import deepcopy
from statistics import mean

from .data import (angka_valid, baca_data, inisialisasi_acak,
                   nilai_objektif, periksa_state, ukuran)
from .pencatatan import simpan_percobaan, ukur_pencarian


def buat_populasi(kapal, kendaraan, state_awal, jumlah_populasi, acak):
    populasi = [deepcopy(state_awal)]
    for _ in range(jumlah_populasi - 1):
        populasi.append(inisialisasi_acak(kapal, kendaraan,
acak.getrandbits(64)))
    return populasi


def pilih_induk(populasi, nilai, acak):
    # Semua fitness nol berarti setiap individu punya peluang yang sama.
    return acak.choices(populasi, weights=nilai if any(nilai) else None, k=2)


def perbaiki(kapal, kendaraan, state, acak):
    hasil = {k["id"]: None for k in kendaraan}
    urutan = list(kendaraan)
    acak.shuffle(urutan)
    for k in urutan:
        hasil[k["id"]] = deepcopy(state[k["id"]])
        try:
            periksa_state(kapal, kendaraan, hasil)
        except ValueError:
            hasil[k["id"]] = None
    return hasil
```

```
def kawin_silang(kapal, kendaraan, induk_a, induk_b, acak):
    if len(kendaraan) < 2:
        return deepcopy(induk_a), deepcopy(induk_b)
    titik = acak.randrange(1, len(kendaraan))
    anak = []
    for kiri, kanan in ((induk_a, induk_b), (induk_b, induk_a)):
        gabungan = {k["id"]: (kiri if i < titik else kanan)[k["id"]]
                    for i, k in enumerate(kendaraan)}
        anak.append(perbaiki(kapal, kendaraan, gabungan, acak))
    return tuple(anak)


def mutasi(kapal, kendaraan, state, acak, peluang_mutasi):
    hasil = deepcopy(state)
    if not kendaraan or acak.random() >= peluang_mutasi:
        return hasil
    k = acak.choice(kendaraan)
    lama = hasil[k["id"]]
    pilihan = [None]
    dek = kapal[0]
    for orientasi in ("vertikal", "horizontal"):
        lebar, panjang = ukuran(k, orientasi)
        for x in range(dek["lebar"] - lebar + 1):
            for y in range(dek["panjang"] - panjang + 1):
                pilihan.append(dict(kapal_id=dek["id"], x=x, y=y,
orientasi=orientasi))
    acak.shuffle(pilihan)
    for posisi in pilihan:
        if posisi == lama:
            continue
        hasil[k["id"]] = posisi
        try:
            periksa_state(kapal, kendaraan, hasil)
            return hasil
        except ValueError:
            continue
    hasil[k["id"]] = lama
    return hasil


def cari(kapal, kendaraan, state_awal, seed,
         jumlah_populasi=20, batas_iterasi=100, peluang_mutasi=0.2):
    if type(seed) is not int:
```

```
        raise ValueError("seed harus integer")
    if type(jumlah_populasi) is not int or jumlah_populasi < 2:
        raise ValueError("jumlah_populasi minimal 2")
    if type(batas_iterasi) is not int or batas_iterasi < 0:
        raise ValueError("batas_iterasi harus integer nonnegatif")
    if not angka_valid(peluang_mutasi) or peluang_mutasi > 1:
        raise ValueError("peluang_mutasi harus antara 0 dan 1")
    nilai_objektif(kapal, kendaraan, state_awal)
    acak = random.Random(seed)
    populasi = buat_populasi(kapal, kendaraan, state_awal, jumlah_populasi, acak)
    populasi_awal = deepcopy(populasi)
    riwayat = []

    for iterasi in range(batas_iterasi + 1):
        nilai = [nilai_objektif(kapal, kendaraan, p) for p in populasi]
        terbaik = populasi[nilai.index(max(nilai))]
        riwayat.append(dict(iterasi=iterasi, maksimum_populasi=max(nilai),
                            rata_rata_populasi=mean(nilai)))
        if iterasi == batas_iterasi:
            break
        generasi_baru = [deepcopy(terbaik)]
        while len(generasi_baru) < jumlah_populasi:
            induk_a, induk_b = pilih_induk(populasi, nilai, acak)
            for anak in kawin_silang(kapal, kendaraan, induk_a, induk_b, acak):
                generasi_baru.append(mutasi(kapal, kendaraan, anak, acak,
peluang_mutasi))
                if len(generasi_baru) == jumlah_populasi:
                    break
        populasi = generasi_baru

    return deepcopy(terbaik), riwayat, {
        "populasi_awal": populasi_awal, "populasi_akhir": deepcopy(populasi),
        "parameter_efektif": dict(jumlah_populasi=jumlah_populasi,
                                  batas_iterasi=batas_iterasi,
peluang_mutasi=peluang_mutasi)}


def main():
    from .visualisasi import simpan_gambar

    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--kapal", default="data/kapal.json")
    parser.add_argument("--kendaraan", default="data/kendaraan.json")
    parser.add_argument("--output", default="output/genetika_42")
```

```
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--jumlah-populasi", type=int, default=20)
    parser.add_argument("--batas-iterasi", type=int, default=100)
    parser.add_argument("--peluang-mutasi", type=float, default=0.2)
    args = parser.parse_args()
    parameter = dict(jumlah_populasi=args.jumlah_populasi,
                     batas_iterasi=args.batas_iterasi,
peluang_mutasi=args.peluang_mutasi)
    kapal, kendaraan = baca_data(args.kapal, args.kendaraan)
    awal = inisialisasi_acak(kapal, kendaraan, args.seed)
    hasil, durasi = ukur_pencarian(
        cari, kapal, kendaraan, awal, args.seed, **parameter)
    akhir, riwayat, metrik = hasil
    awal = max(metrik["populasi_awal"],
               key=lambda p: nilai_objektif(kapal, kendaraan, p))
    tujuan = simpan_percobaan(
        args.output, algoritma="genetic_algorithm", seed=args.seed,
parameter=parameter,
        kapal=kapal, kendaraan=kendaraan, state_awal=awal, state_akhir=akhir,
        durasi_detik=durasi, riwayat=riwayat, metrik=metrik)
    simpan_gambar(tujuan)
    print(f"Nilai: {riwayat[0]['maksimum_populasi']:.2f} -> "
          f"{riwayat[-1]['maksimum_populasi']:.2f}, durasi: {durasi:.3f} detik")
    print(f"Hasil tersimpan: {tujuan}")


if __name__ == "__main__":
    main()
```

2.2.5 Pencatatan dan Visualisasi

Hasil setiap run disimpan sebagai JSON dan CSV melalui pencatatan.py. Durasi hanya mengukur fungsi pencarian, termasuk pembentukan populasi GA. Pembacaan data, state acak pertama, penyimpanan, dan pembuatan gambar berada di luar timer. Ringkasan beberapa pengulangan dibuat oleh runner eksperimen setelah seluruh run selesai.

Visualisasi menggunakan Matplotlib dalam visualisasi.py. Kendaraan digambar sebagai persegi panjang sesuai posisi dan orientasinya, dilengkapi objective, berat muatan, dan daftar kendaraan di luar kapal. Grafik HC dan SA menampilkan objective terhadap iterasi, sedangkan GA menampilkan maksimum dan rata-rata fitness populasi.

Fungsi simpan_gambar() membaca hasil.json, kemudian menyimpan gambar state awal, state akhir, dan grafik objective sebagai PNG. Jika tersedia data eksponen SA, fungsi ini juga membuat penerimaan_sa.png dengan skala symlog agar nilai besar tetap terbaca.

Tabel 8. Berkas keluaran eksperimen

File	Isi
hasil.json	Dataset, parameter, state, objektif, durasi, dan metrik
riwayat.csv	Nilai per iterasi
state_awal.png, state_akhir.png	Penempatan kendaraan
objektif.png	Kurva objektif / statistik populasi algoritma genetic
penerimaan_sa.png	eksponen penerimaan simulated annealing
ringasan.csv, ringkasan.md	ringkasan beberapa pengulangan

Tabel 9. Fungsi visualisasi

Kelas/ Fungsi	Deskripsi
gambar_state()	Menggambar dek kapal, kendaraan, objektif, berat, dan kendaraan di luar kapal
gambar_riwayat()	Menggambar objektif HC / SA atau statistik populasi GA
gambar_penerimaan()	Menggambar eksponen penerimaan SA
simpan_gambar()	Membaca hasil JSON dan menyimpan seluruh gambar PNG

```
import textwrap
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

from .data import OBJEKTIF, baca_json, nilai_objektif, ukuran
from .pencatatan import periksa_riwayat


def gambar_state(kapal, kendaraan, state, judul="Keadaan kapal",
objektif="tarif_kirim"):
   nilai = nilai_objektif(kapal, kendaraan, state, objektif)
   satuan = OBJEKTIF[objektif][1]
   kendaraan_luar = [k["id"] for k in kendaraan if state[k["id"]] is None]
   catatan = "Di luar kapal: " + (", ".join(kendaraan_luar) or "Tidak ada")
   baris_catatan = textwrap.wrap(catatan, width=100)
   fig, axes = plt.subplots(1, len(kapal), squeeze=False,
                            figsize=(max(7, 6 * len(kapal)), 8))
   warna = {k["id"]: plt.get_cmap("tab20")(i % 20) for i, k in
enumerate(kendaraan)}

   for dek, ax in zip(kapal, axes[0]):
       berat = 0
       for data_kendaraan in kendaraan:
           penempatan = state[data_kendaraan["id"]]
           if penempatan is None or penempatan["kapal_id"] != dek["id"]:
               continue
           lebar, panjang = ukuran(data_kendaraan, penempatan["orientasi"])
           x, y = penempatan["x"], penempatan["y"]
           berat += data_kendaraan["berat"]
           ax.add_patch(Rectangle(
               (x, y), lebar, panjang, facecolor=warna[data_kendaraan["id"]],
               edgecolor="black", alpha=.85))
           label = f"{data_kendaraan['id']}\n{penempatan['orientasi']}"
           ax.text(x + lebar / 2, y + panjang / 2, label, ha="center",
va="center",
                   fontsize=7, rotation=90 if lebar == 1 else 0)
       ax.set(xlim=(0, dek["lebar"]), ylim=(dek["panjang"], 0),
              xlabel="x (sel) →", ylabel="y (sel) ↓",
```

```
                title=f"{dek['id']}\nBerat {berat:g}/{dek['kapasitas']:g} unit")
        ax.set_xticks(range(dek["lebar"] + 1))
        ax.set_yticks(range(dek["panjang"] + 1))
        ax.grid(alpha=.25)
        ax.set_aspect("equal")
    fig.suptitle(f"{judul}\nObjektif: {nilai:g} {satuan}")
    fig.text(.04, .02, "\n".join(baris_catatan), fontsize=9, va="bottom")
    fig.tight_layout(rect=(0, .06 + .026 * len(baris_catatan), 1, .91))
    return fig


def gambar_riwayat(riwayat, judul="Perkembangan objektif",
objektif="tarif_kirim"):
    if objektif not in OBJEKTIF:
        raise ValueError("objektif harus tarif_kirim atau tarif_eta")
    periksa_riwayat(riwayat)
    fig, ax = plt.subplots(figsize=(8, 4))
    seri = (("objektif", "Objektif state saat ini"),
            ("maksimum_populasi", "Maksimum populasi"),
            ("rata_rata_populasi", "Rata-rata populasi"))
    for kolom, label in seri:
        data = [(b["iterasi"], b[kolom]) for b in riwayat if b.get(kolom) is not
None]
        if data:
            ax.plot(*zip(*data), label=label, marker="o", markersize=3)
    ax.set(xlabel="Iterasi", ylabel=f"Objektif ({OBJEKTIF[objektif][1]})",
title=judul)
    ax.legend()
    ax.grid(alpha=.25)
    fig.tight_layout()
    return fig


def simpan_gambar(direktori):
    """Membuat PNG dari hasil percobaan yang sudah disimpan."""
    tujuan = Path(direktori)
    hasil = baca_json(tujuan / "hasil.json")
    objektif = hasil["objektif_id"]
    skenario = hasil["skenario"].replace("_", " ")
    for tahap in ("awal", "akhir"):
        fig = gambar_state(hasil["kapal"], hasil["kendaraan"],
hasil[f"state_{tahap}"],
                           judul=f"State {tahap} | {skenario}", objektif=objektif)
        fig.savefig(tujuan / f"state_{tahap}.png", dpi=150)
```

```
        plt.close(fig)
    fig = gambar_riwayat(hasil["riwayat"], objektif=objektif)
    fig.savefig(tujuan / "objektif.png", dpi=150)
    plt.close(fig)
    if any(b.get("eksponen_penerimaan") is not None for b in hasil["riwayat"]):
        fig = gambar_penerimaan(hasil["riwayat"])
        fig.savefig(tujuan / "penerimaan_sa.png", dpi=150)
        plt.close(fig)


def gambar_penerimaan(riwayat, judul="Eksponen penerimaan SA"):
    periksa_riwayat(riwayat)
    data = [(b["iterasi"], b["eksponen_penerimaan"])
            for b in riwayat if b.get("eksponen_penerimaan") is not None]
    if not data:
        raise ValueError("Riwayat tidak memiliki eksponen_penerimaan SA")
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.plot(*zip(*data), marker="o", markersize=3)
    ax.set(xlabel="Iterasi", ylabel="exp(ΔE/T)", title=judul)
    ax.grid(alpha=.25)
    fig.tight_layout()
    return fig
```

2.3 Hasil Eksperimen

2.3.1 Konfigurasi Eksperimen

Eksperimen dijalankan lokal menggunakan satu dataset berisi 30 kendaraan. Kapal berukuran 10 x 16 sel dengan kapasitas 60 satuan unit. Objektif adalah total tarif kirim kendaraan yang dimuat, dalam juta rupiah. Setiap konfigurasi diulang dengan seed 40, 41, dan 42. Dataset tidak diubah sama sekali.

Tabel 10. Konfigurasi parameter eksperimen

Algoritma	Parameter
Stochastic Hill Climbing	maks_gagal=1000
Simulated Annealing	suhu_awal=50 , pendinginan=0.995, suhu_minimum=0.1, batas_iterasi=1000
	evaluation_bias=1, interval_stuck=50
Genetic Algorithm 1	populasi 10, iterasi {1,10,100} , peluang_mutasi=0.2

Genetic Algorithm 2	generasi 30, populasi {2,10,30} individu, peluang_mutasi = 0.2
[Bonus] Steepest HC	berhenti ketika tidak ada *state* yang lebih bagus dari current
[Bonus] HC + Sideways move	maks_sideways = 100
[Bonus] Random restart	maks_restart = 10

Rentang iterasi dibuat lebar untuk mengamati tahap awal dan perkembangan pencarian. Semua hasil, termasuk konfigurasi yang tidak membaik, dilaporkan. Seluruh lokasi gambar di bawah relatif terhadap root repository IF3070_G34_TIRTA.

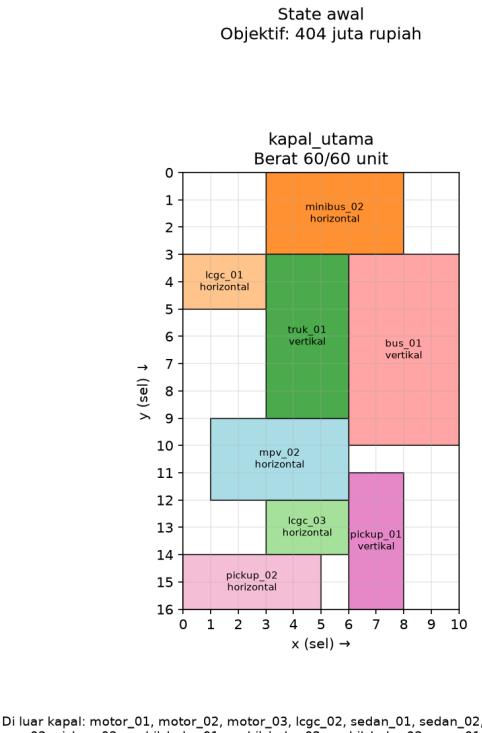
2.3.2 Eksperimen Hill-Climbing

HC wajib menggunakan stochastic. Objektif meningkat dari 404, 412, dan 414 menjadi 410, 421, dan 425. Rata-rata objective akhir adalah 418.667 juta rupiah dengan rata-rata durasi 0.882485 detik.

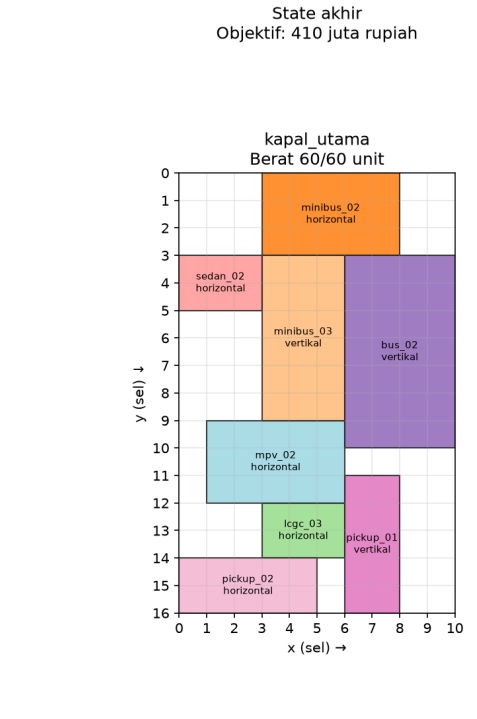
Tabel 11. Hasil tiga pengulangan stochastic Hill-Climbing

Run	Seed	Nilai OF Awal	Nilai OF Akhir	Banyak Iterasi	Durasi (s)
1	40	404	*410*	*1125*	*0.558499*
2	*41*	*412*	*421*	*2259*	*1.277588*
3	*42*	*414*	*425*	*1635*	*0.811369*

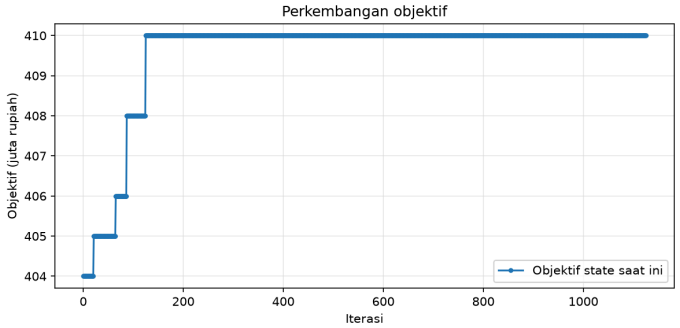
Run 1 Seed 40



Gambar 1. State awal stochastic Hill-Climbing (run 1, seed 40)

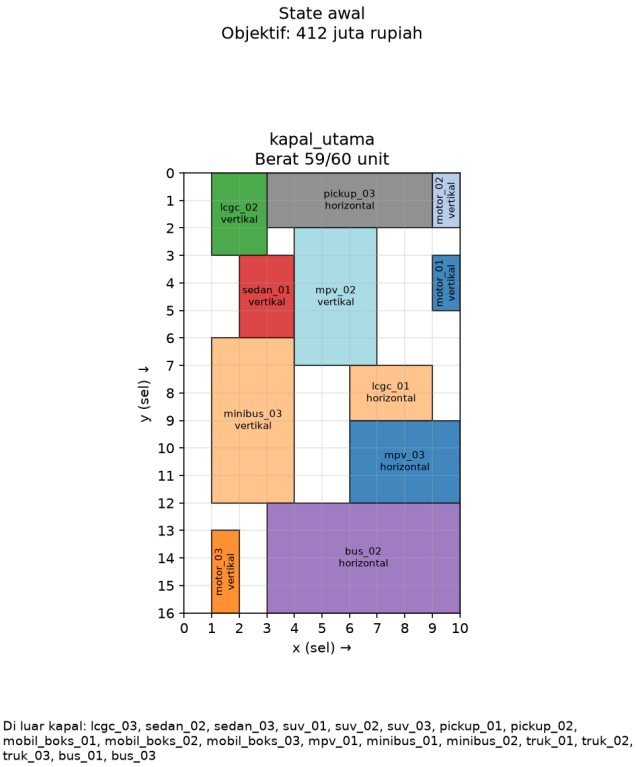


Gambar 2. State akhir stochastic Hill-Climbing (run 1, seed 40)

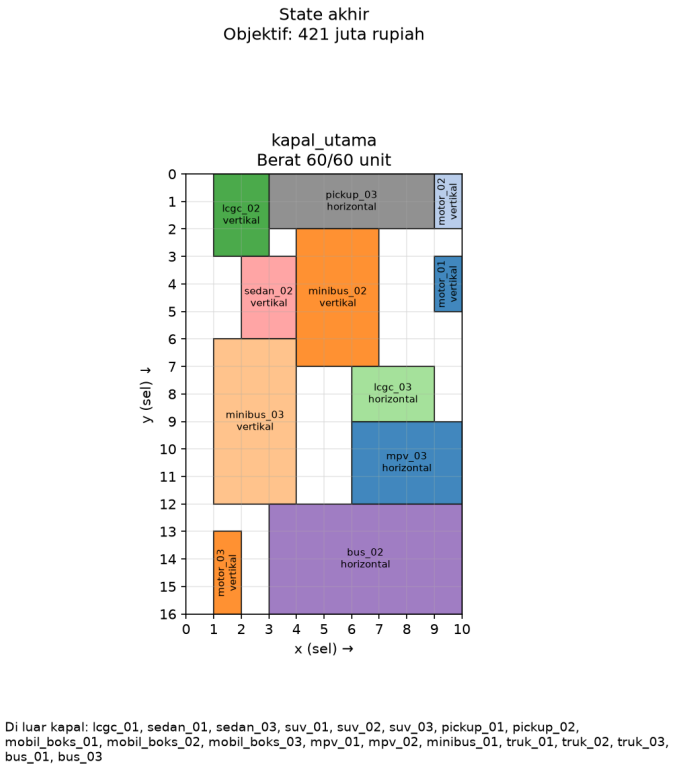


Gambar 3. Kurva objektif stochastic Hill-Climbing (run 1, seed 40)

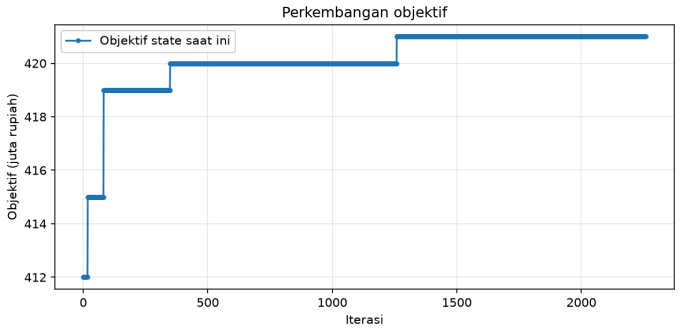
Run 2 Seed 41



Gambar 4. State awal stochastic Hill-Climbing (run 2, seed 41)

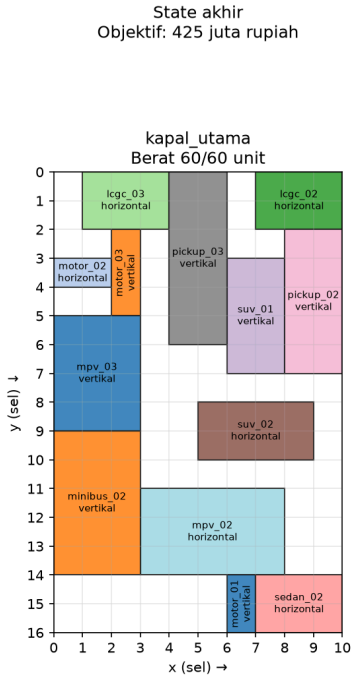


Gambar 5. State akhir stochastic Hill-Climbing (run 2, seed 41)



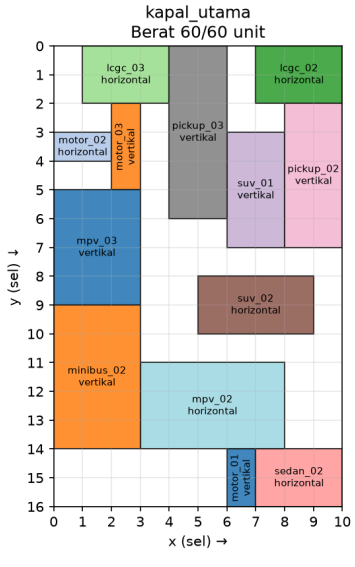
Gambar 6. Kurva objektif stochastic Hill-Climbing (run 2, seed 41)

Run 3 Seed 42



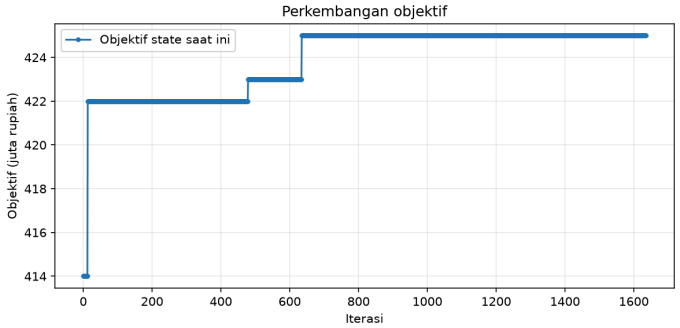
Di luar kapal: lcgc_01, sedan_01, sedan_03, suv_03, pickup_01, mobil_boks_01, mobil_boks_02, mobil_boks_03, mpv_01, minibus_01, minibus_03, truk_01, truk_02, truk_03, bus_01, bus_02, bus_03

Gambar 7. State awal stochastic Hill-Climbing (run 3, seed 42)



Di luar kapal: lcgc_01, sedan_01, sedan_03, suv_03, pickup_01, mobil_boks_01, mobil_boks_02, mobil_boks_03, mpv_01, minibus_01, minibus_03, truk_01, truk_02, truk_03, bus_01, bus_02, bus_03

Gambar 8. State akhir stochastic Hill-Climbing (run 3, seed 42)



Gambar 9. Kurva objektif stochastic Hill-Climbing (run 3, seed 42)

[Bonus]

Steepest menghasilkan rata-rata 418.333 juta rupiah. Sideways menghasilkan 419.333 juta rupiah dengan 100 langkah sideways pada setiap run. Random restart menghasilkan 426.000 juta rupiah. Batas sepuluh restart menghasilkan sebelas pencarian per run.

Tabel 12. [BONUS] Hasil eksperimen varian Hill-Climbing

Varian	Run	Seed	Nilai OF Awal	Nilai OF Akhir	Indeks Iterasi Akhir	Durasi (s)
HC steepest	1	40	404	*409*	*3*	*0.051673*
	2	*41*	*412*	*421*	*3*	*0.082582*
	3	*42*	*414*	*425*	*3*	*0.079347*
HC sideways	1	*40*	*404*	*409*	*103*	*1.323967*
	2	*41*	*412*	*424*	*104*	*1.682145*
	3	*42*	*414*	*425*	*103*	*1.776858*
HC random restart	1	*40*	*404*	*429*	*54*	*0.816994*
	2	*41*	*412*	*423*	*46*	*0.777754*
	3	*42*	*414*	*426*	*54*	*0.891504*

Tabel 13. [BONUS] Jumlah langkah sideways tiap pengulangan

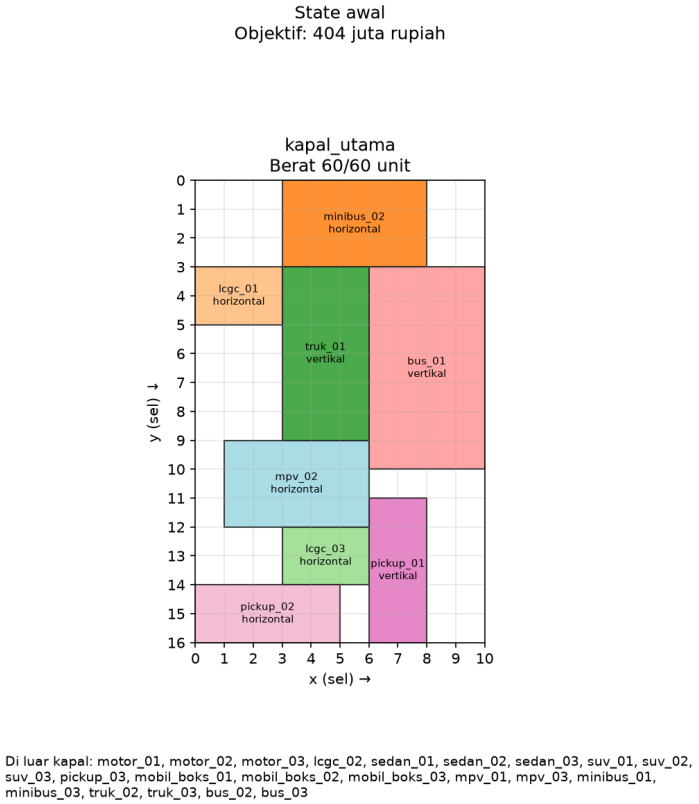
Run	Seed	Batas Sideways	Jumlah Sideways
1	40	*100*	*100*
2	*41*	*100*	*100*
3	*42*	*100*	*100*

Tabel 14. [BONUS] Jumlah restart dan iterasi tiap pencarian

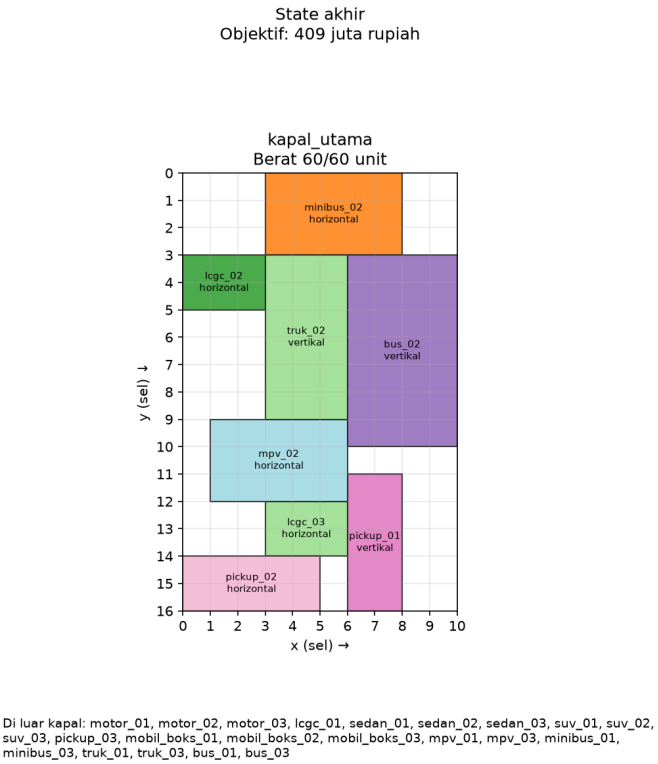
Run	Seed	Jumlah Sideways	Iterasi Run Sampai Restart 10
1	40	*10*	*[3, 4, 2, 5, 1, 3, 4, 3, 7, 5, 7]*
2	*41*	*10*	*[3, 5, 0, 2, 3, 1, 5, 4, 5, 4, 4]*
3	*42*	*10*	*[3, 5, 6, 4, 6, 3, 3, 2, 3, 3, 6]*

Pada random restart, indeks iterasi akhir mencakup sepuluh baris state awal restart. Daftar iterasi per restart hanya menghitung langkah pencarian yang diterima pada setiap pencarian.

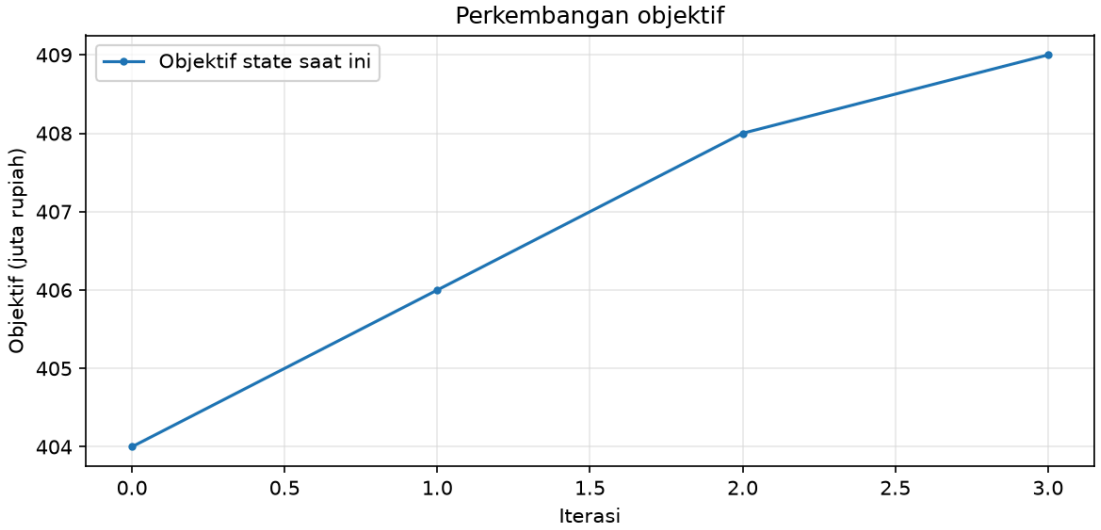
Visualisasi HC Steepest Ascent
Run 1, Seed 40



Gambar 10. State awal steepest ascent (run 1, seed 40)

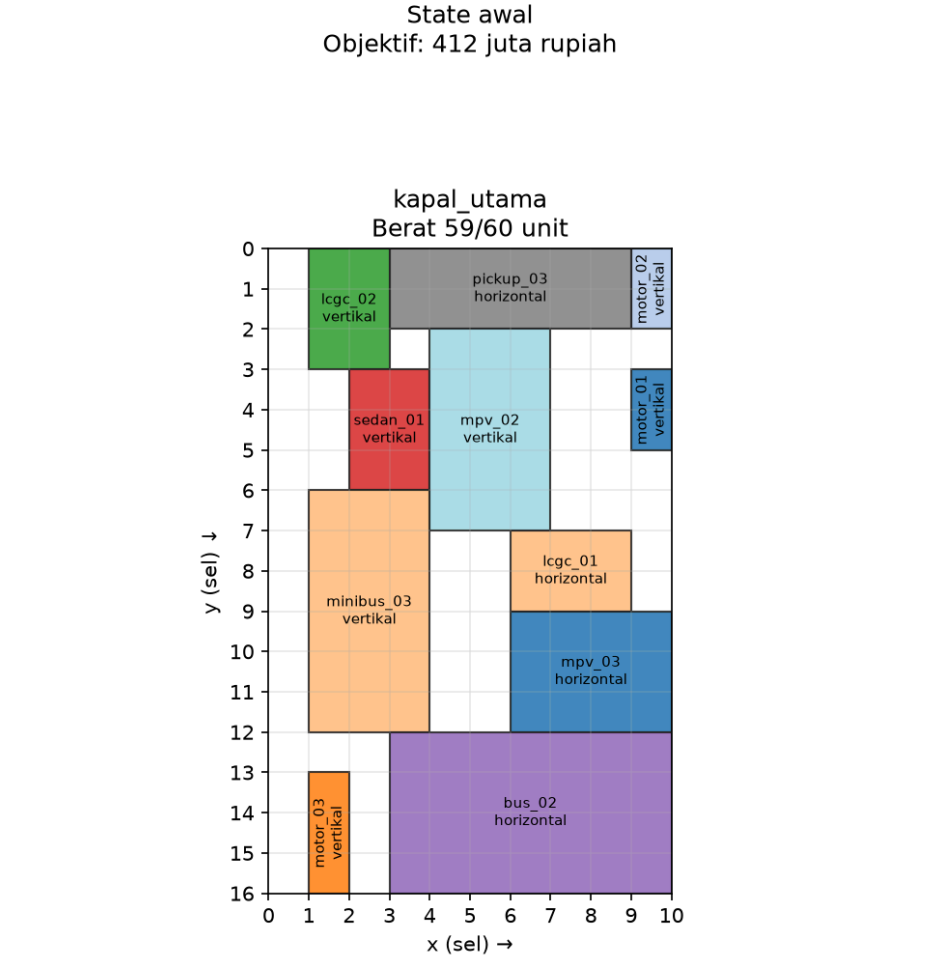


Gambar 11. State akhir steepest ascent (run 1, seed 40)

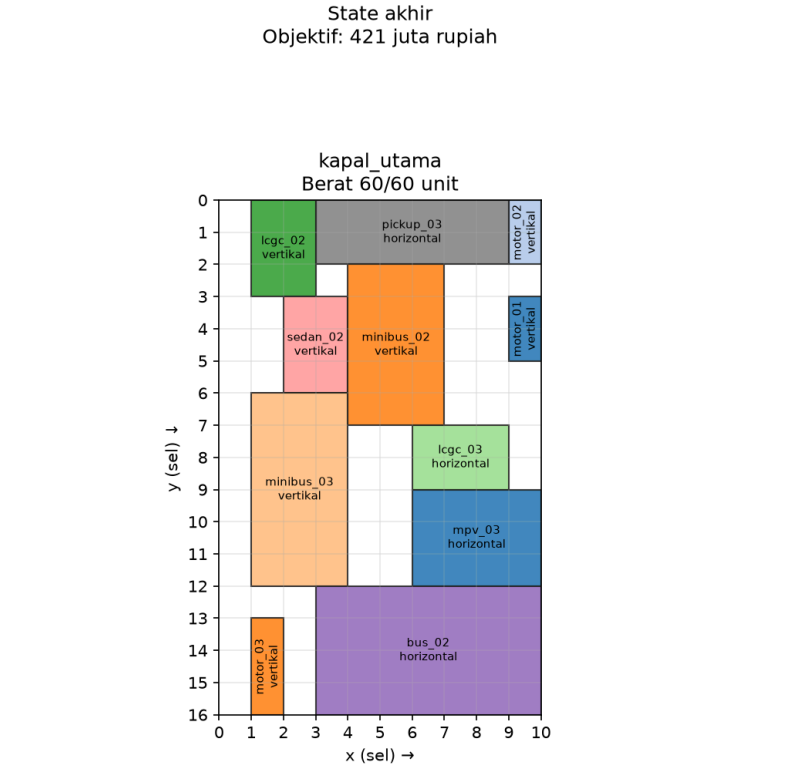


Gambar 12. Kurva objektif steepest ascent (run 1, seed 40)

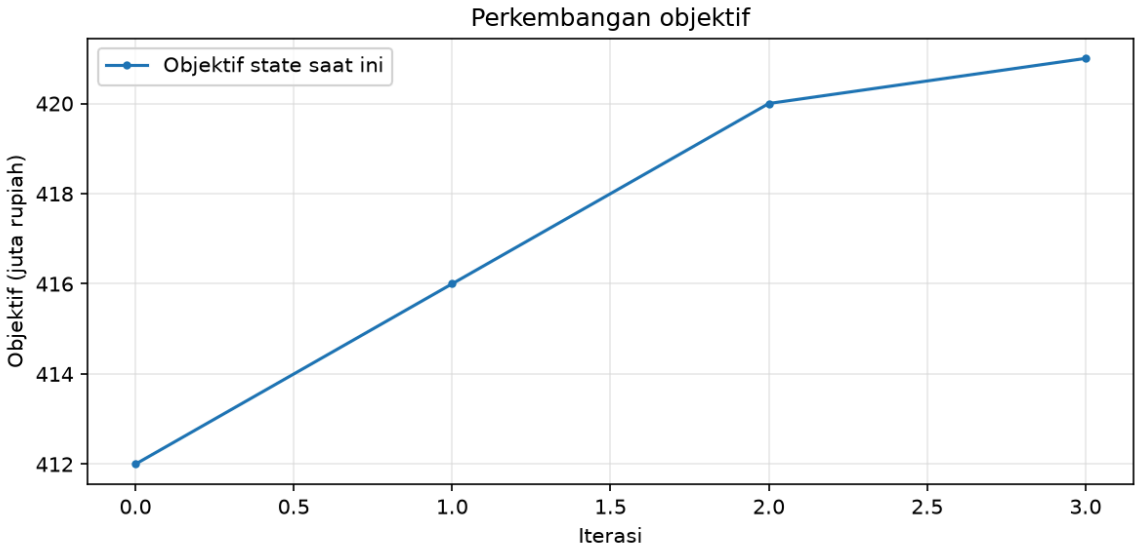
Run 2, Seed 41



Gambar 13. State awal steepest ascent (run 2, seed 41)

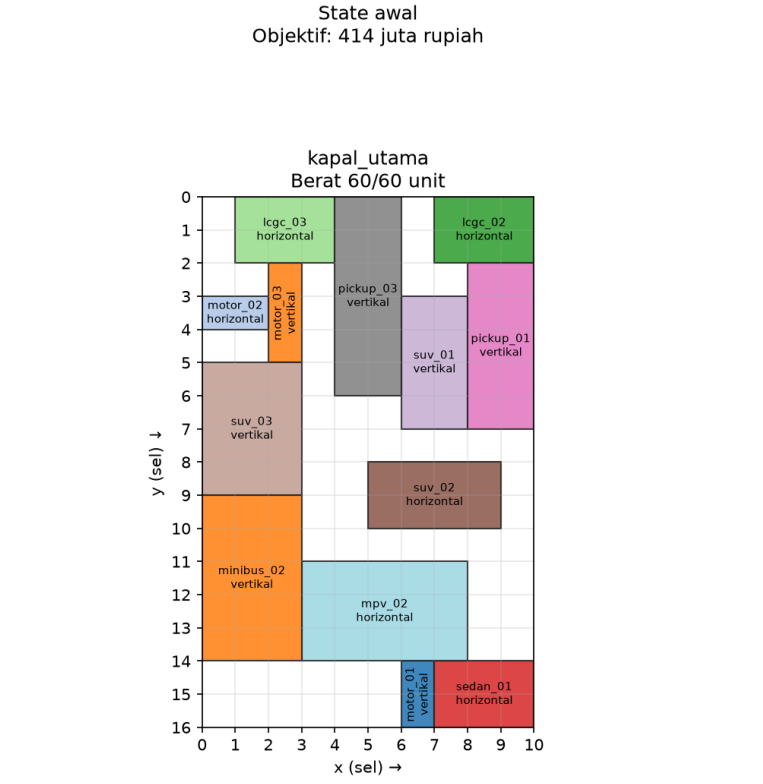


Gambar 14. State akhir steepest ascent (run 2, seed 41)

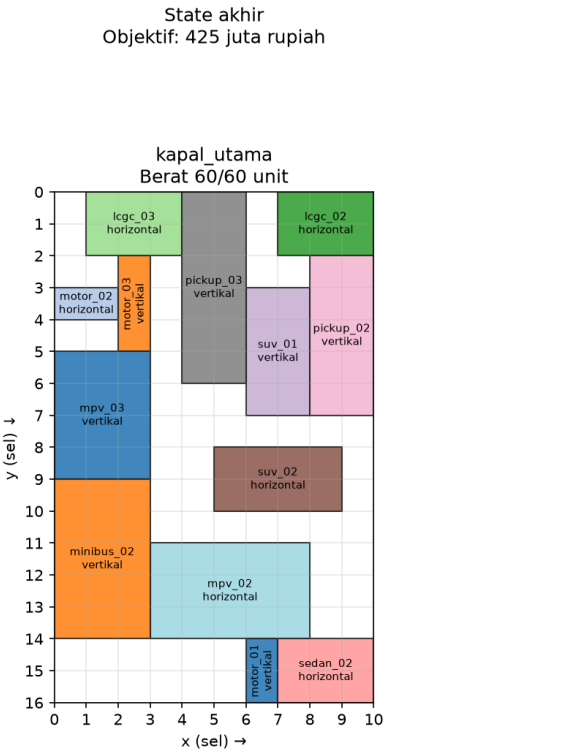


Gambar 15. Kurva objektif steepest ascent (run 2, seed 41)

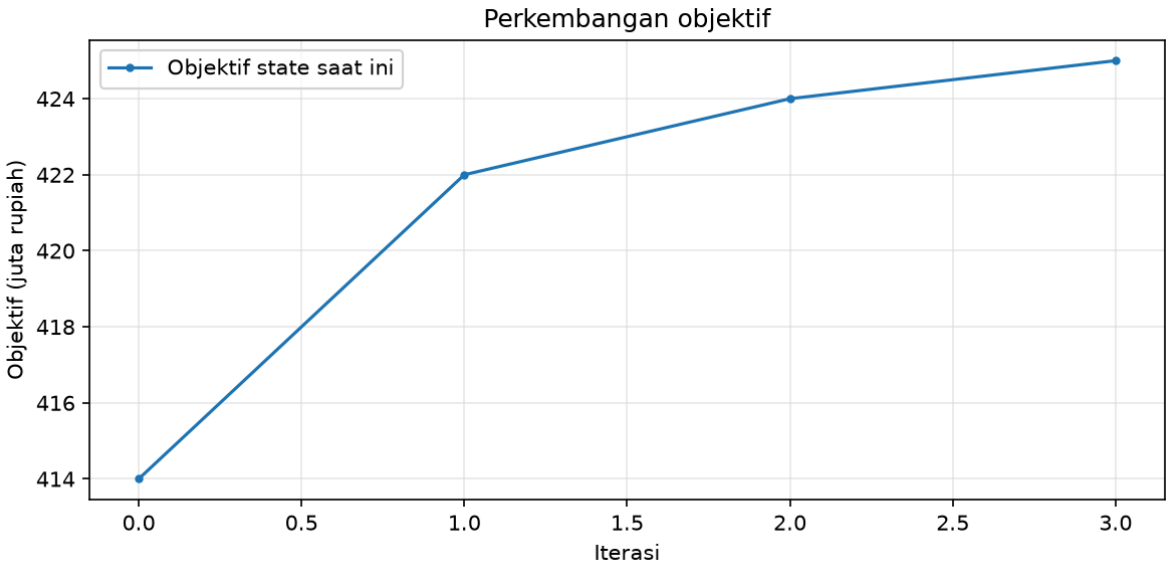
Run 3, Seed 42



Gambar 16. State awal steepest ascent (run 3, seed 42)

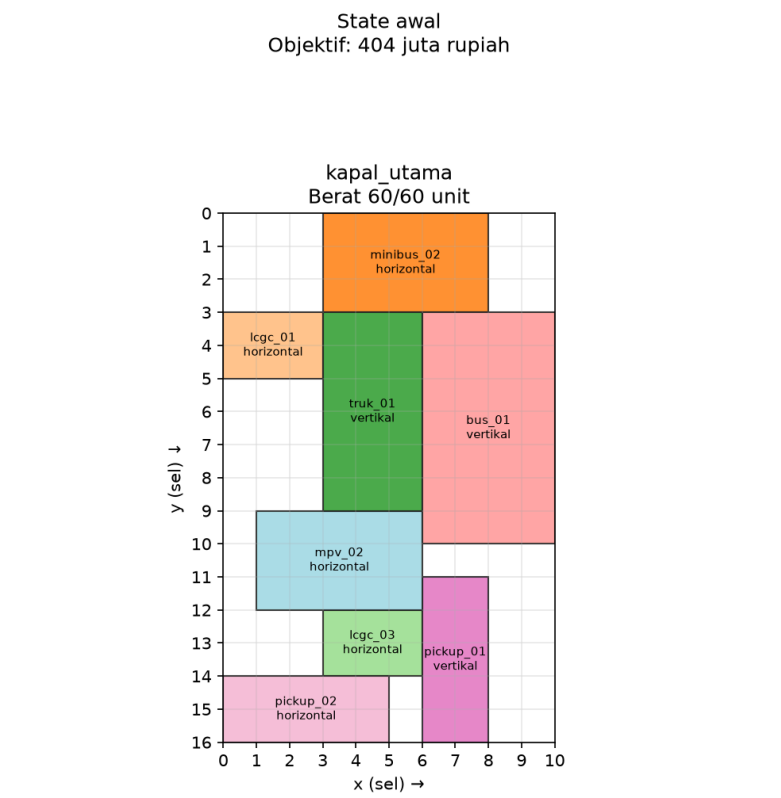


Gambar 17. State akhir steepest ascent (run 3, seed 42)

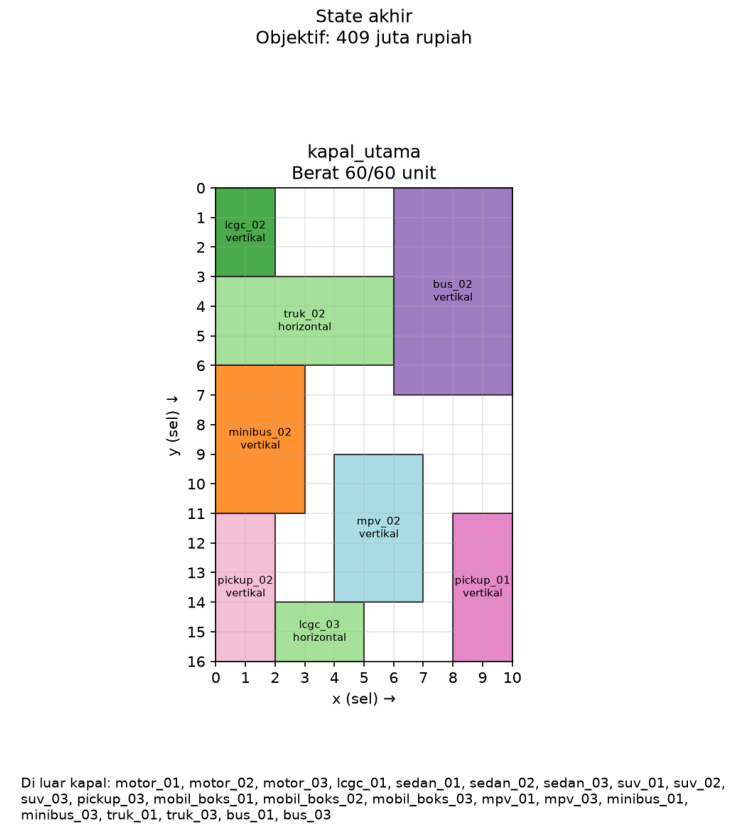


Gambar 18. Kurva objektif steepest ascent (run 3, seed 42)

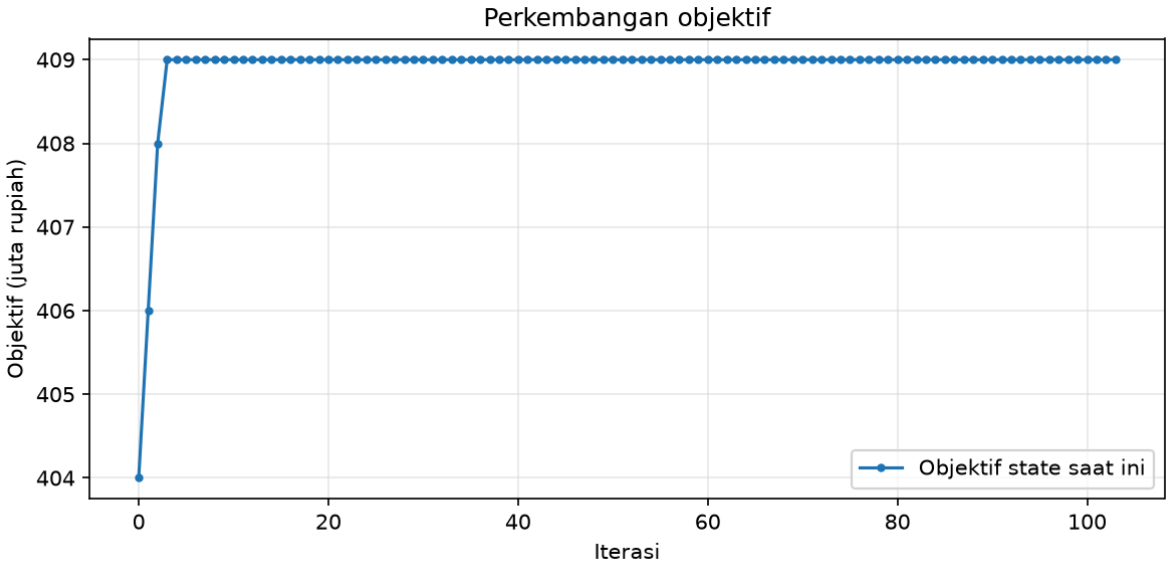
Visualisasi HC + Sideways
Run 1, Seed 40



Gambar 19. State awal sideways move (run 1, seed 40)

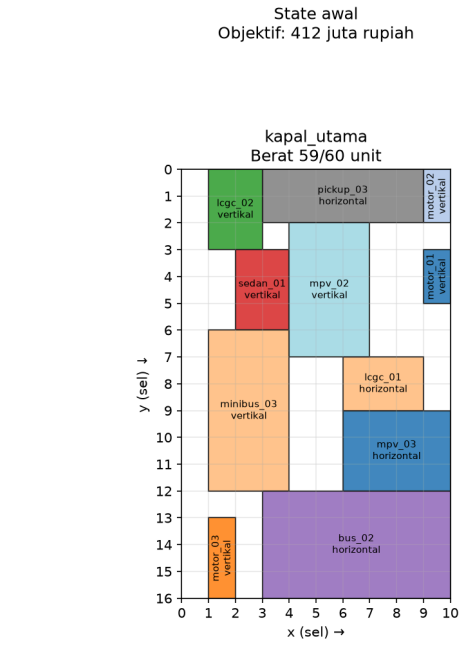


Gambar 20. State akhir sideways move (run 1, seed 40)

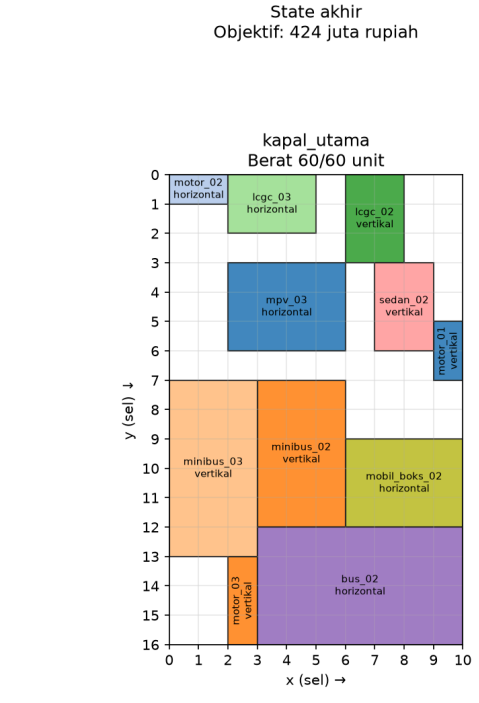


Gambar 21. Kurva objektif sideways move (run 1, seed 40)

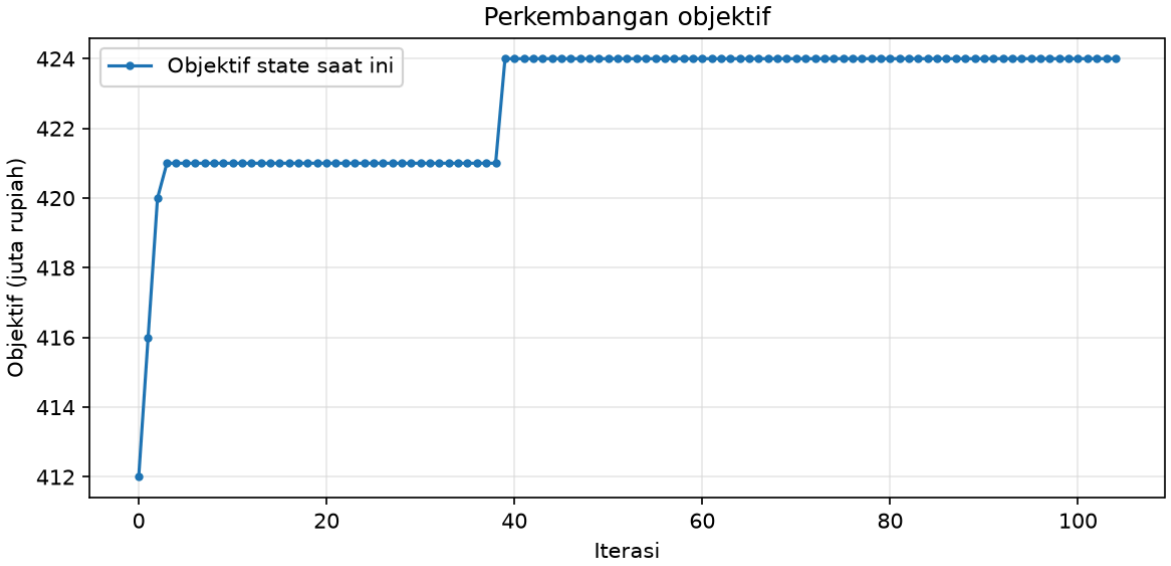
Run 2, Seed 41



Gambar 22. State awal sideways move (run 2, seed 41)

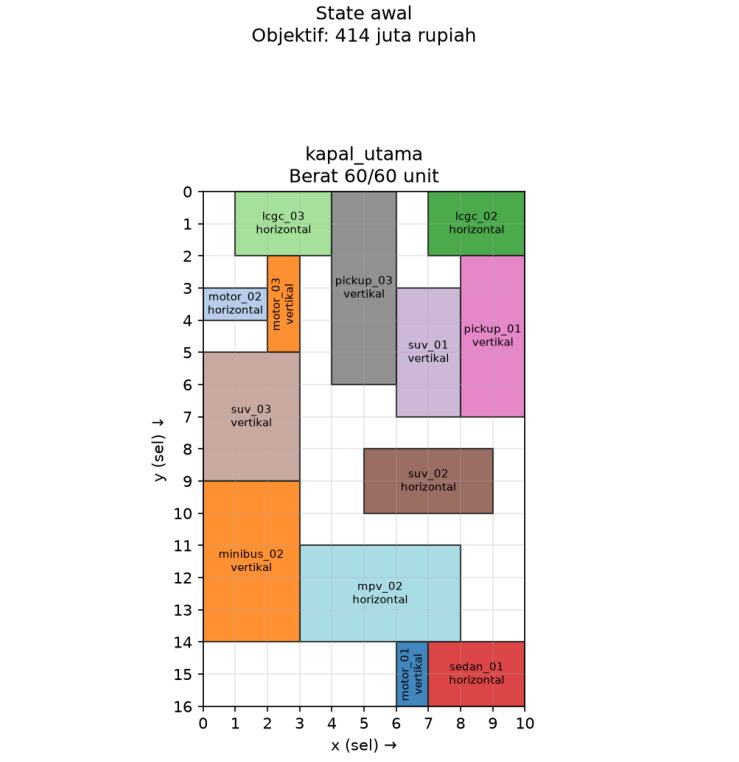


Gambar 23. State akhir sideways move (run 2, seed 41)



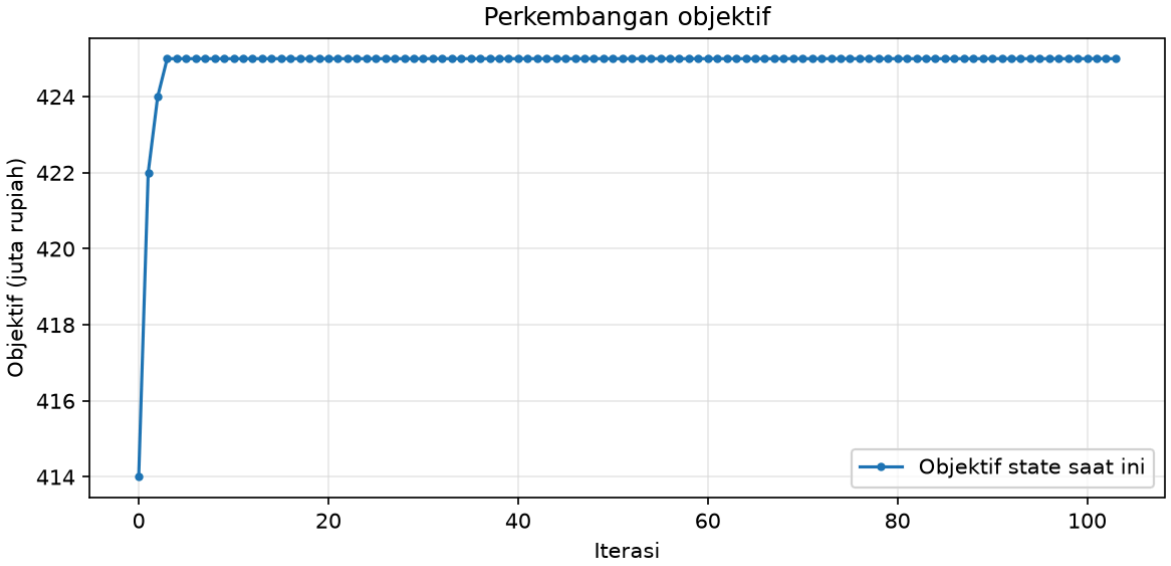
Gambar 24. Kurva objektif sideways move (run 2, seed 41)

Run 3, Seed 42



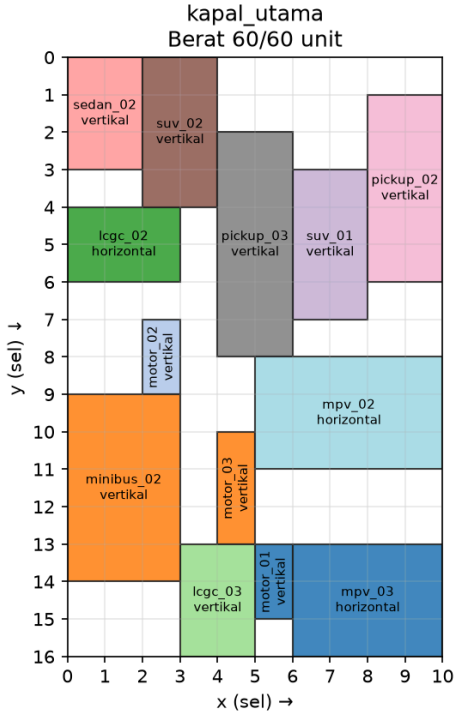
Di luar kapal: lcgc_01, sedan_02, sedan_03, pickup_02, mobil_boks_01, mobil_boks_02, mobil_boks_03, mpv_01, mpv_03, minibus_01, minibus_03, truk_01, truk_02, truk_03, bus_01, bus_02, bus_03

Gambar 25. State awal sideways move (run 3, seed 42)



Gambar 26. Kurva objektif sideways move (run 3, seed 42)

State akhir
Objektif: 425 juta rupiah

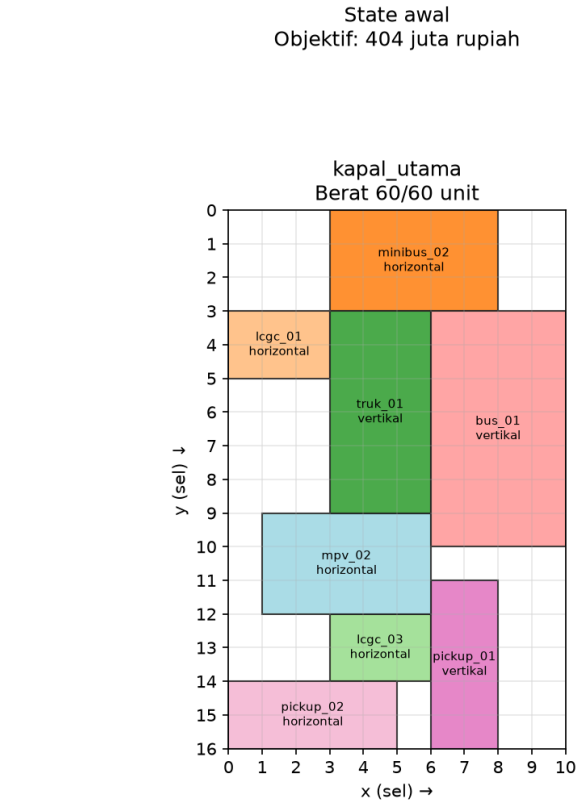


Di luar kapal: lcgc_01, sedan_01, sedan_03, suv_03, pickup_01, mobil_boks_01, mobil_boks_02, mobil_boks_03, mpv_01, minibus_01, minibus_03, truk_01, truk_02, truk_03, bus_01, bus_02, bus_03

Gambar 27. State akhir sideways move (run 3, seed 42)

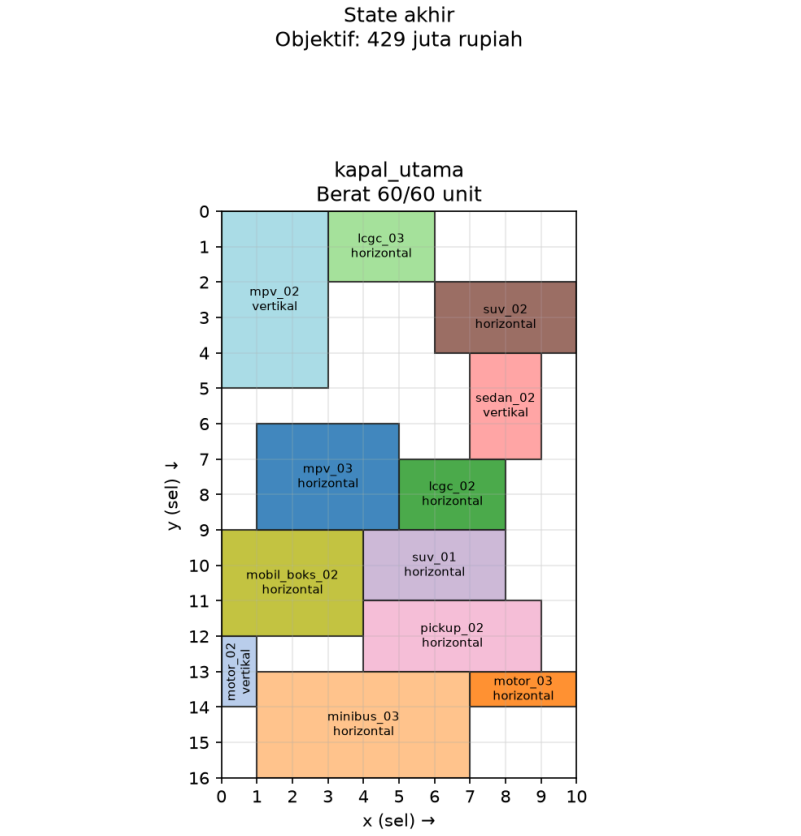
Visualisasi HC with random restart

Run 1, Seed 40

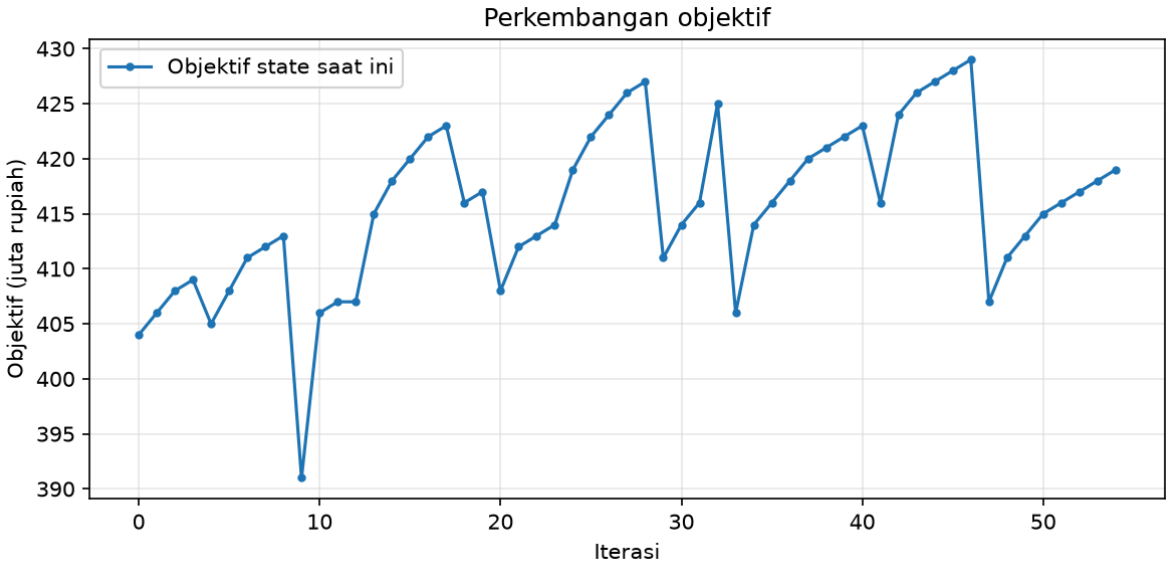


Di luar kapal: motor_01, motor_02, motor_03, lcgc_02, sedan_01, sedan_02, sedan_03, suv_01, suv_02, suv_03, pickup_03, mobil_boks_01, mobil_boks_02, mobil_boks_03, mpv_01, mpv_03, minibus_01, minibus_03, truk_02, truk_03, bus_02, bus_03

Gambar 28. State awal random restart (run 1, seed 40)

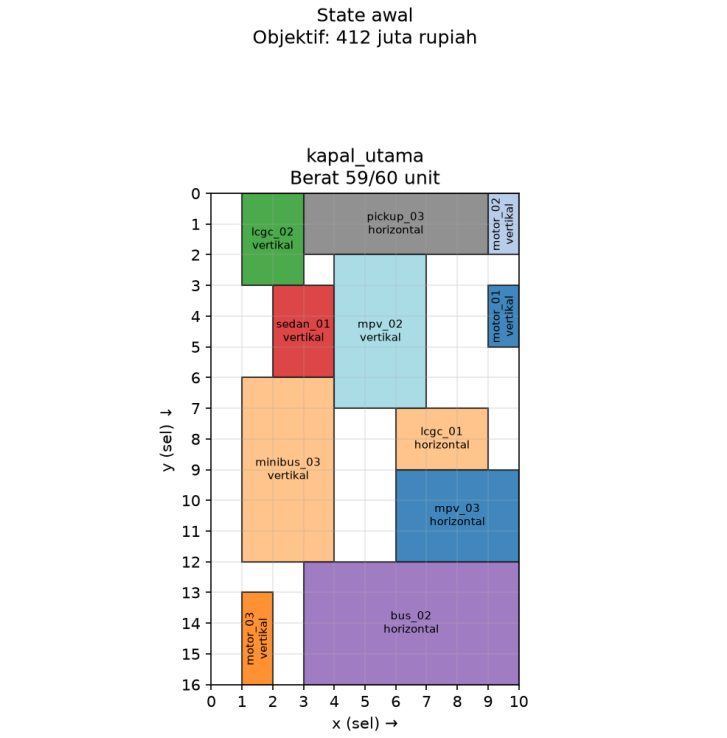


Gambar 29. State terbaik random restart (run 1, seed 40)

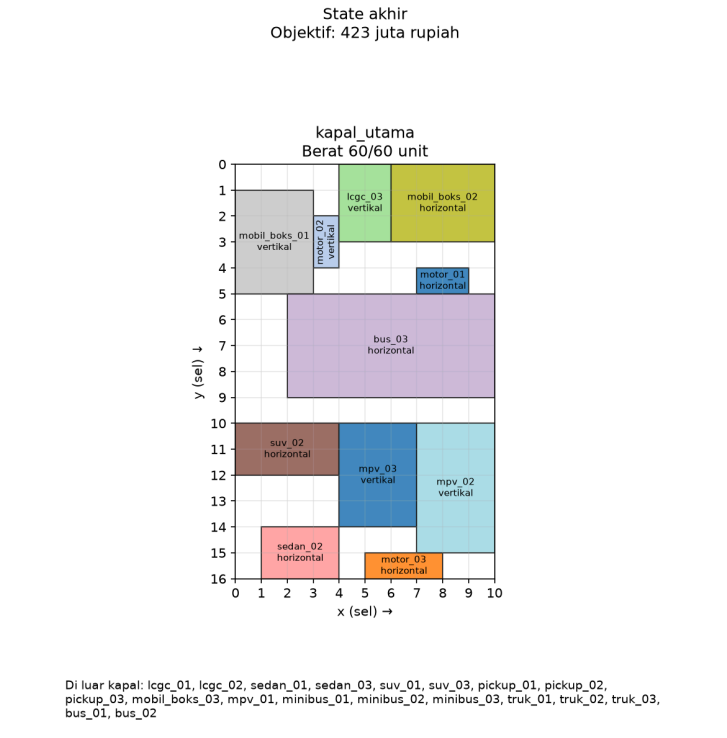


Gambar 30. Kurva objektif random restart (run 1, seed 40)

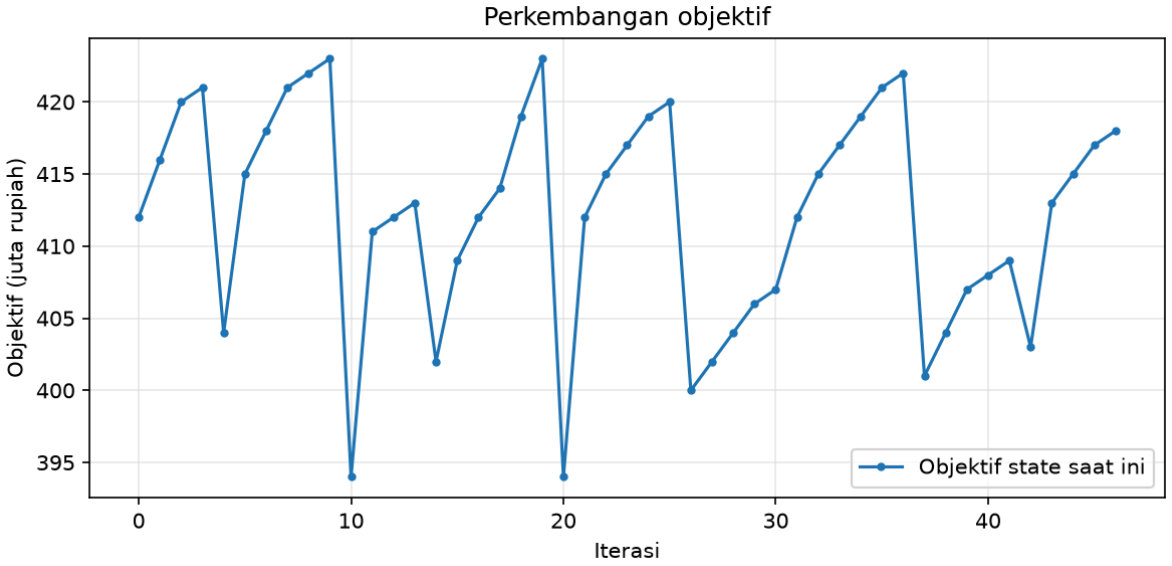
Run 2, Seed 41



Gambar 31. State awal random restart (run 2, seed 41)

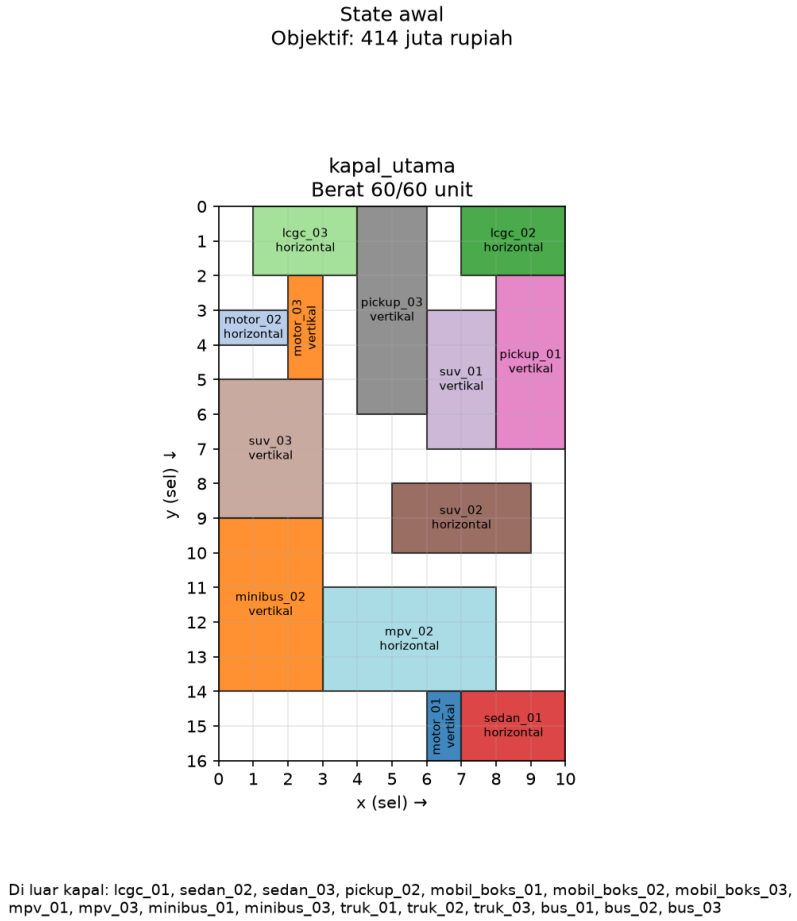


Gambar 32. State terbaik random restart (run 2, seed 41)



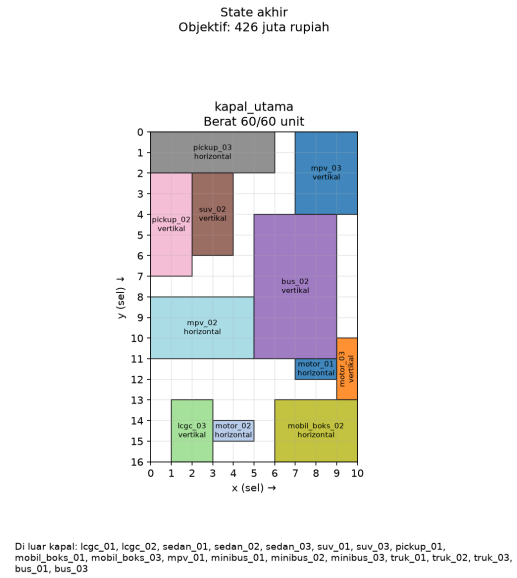
Gambar 33. Kurva objektif random restart (run 2, seed 41)

Run 3, Seed 42

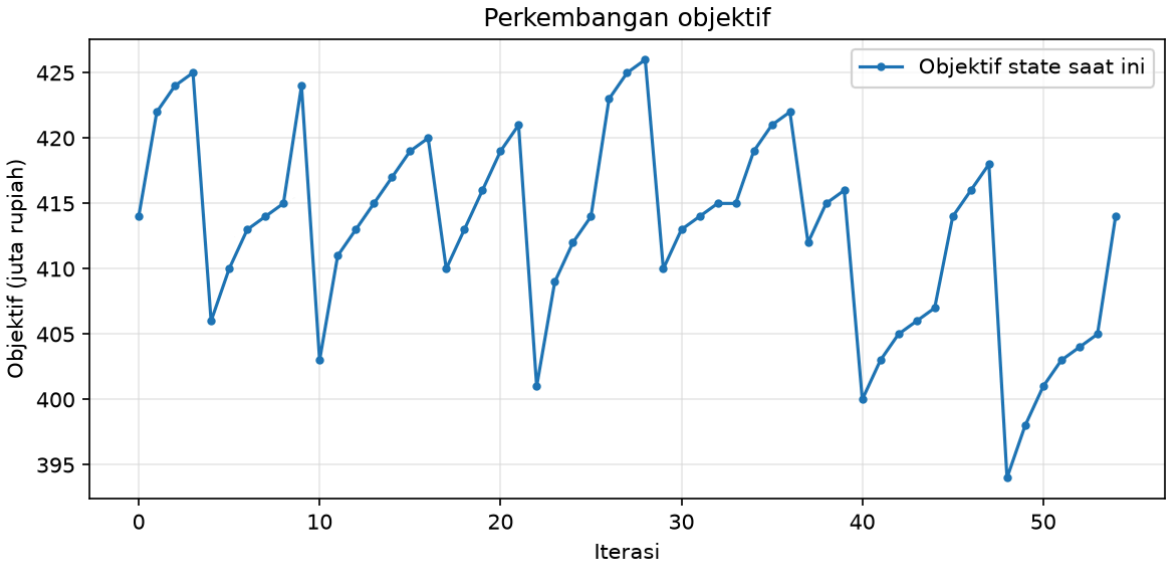


Di luar kapal: lcgc_01, sedan_02, sedan_03, pickup_02, mobil_boks_01, mobil_boks_02, mobil_boks_03, mpv_01, mpv_03, minibus_01, minibus_03, truk_01, truk_02, truk_03, bus_01, bus_02, bus_03

Gambar 34. State awal random restart (run 3, seed 42)



Gambar 35. State terbaik random restart (run 3, seed 42)



Gambar 36. Kurva objektif random restart (run 3, seed 42)

2.3.3 Eksperimen Simulated Annealing

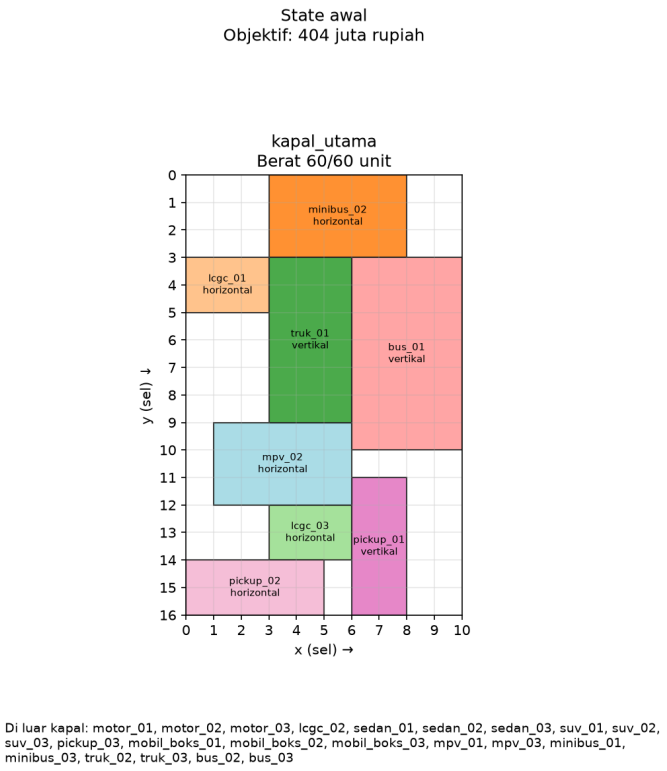
SA menghasilkan objective terbaik 413, 420, dan 417, dengan rata-rata 416.667 juta rupiah dan durasi 0.546154 detik. Ketiga run berhenti pada batas 1.000 iterasi. Seluruh pemanggilan generator memperoleh kandidat valid.

Tabel 15. Hasil tiga pengulangan Simulated Annealing

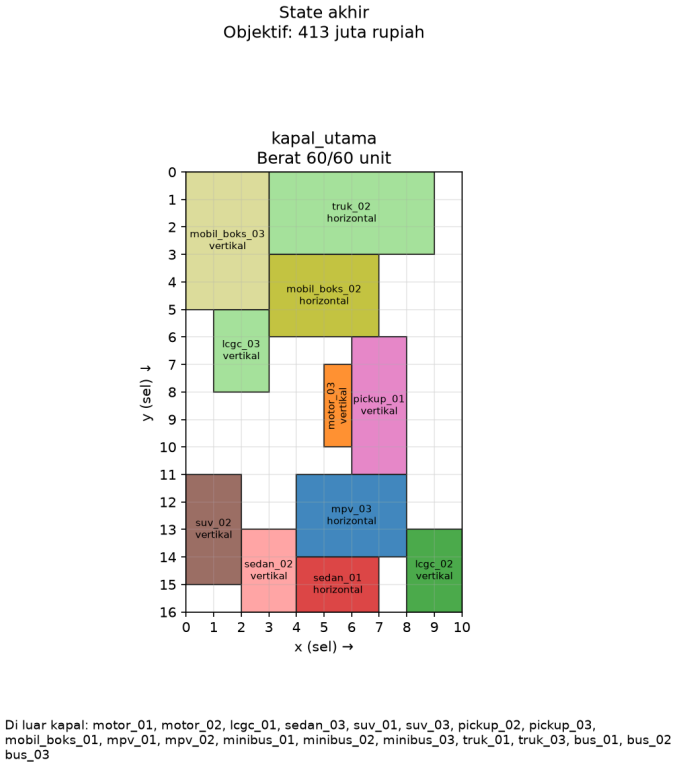
Run	Seed	Nilai OF Awal	Nilai OF Akhir	Frekuensi Stuck	Durasi (s)
1	40	404	413	0%	0.555946
2	41	412	420	0%	0.565304
3	42	414	417	5%	0.517211

Frekuensi stuck masing-masing adalah 0/20, 0/20, dan 1/20 checkpoint. Plateau termasuk dalam definisi stuck. Pada seed 42, titik terakhir grafik bernilai 414, sementara state terbaik yang disimpan bernilai 417.

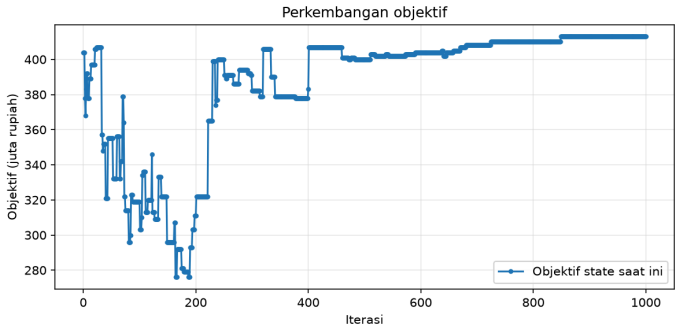
Run 1, Seed 40



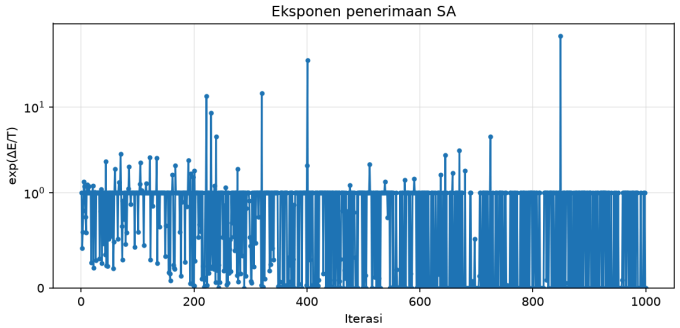
Gambar 37. State awal Simulated Annealing (run 1, seed 40)



Gambar 38. State akhir terbaik Simulated Annealing (run 1, seed 40)

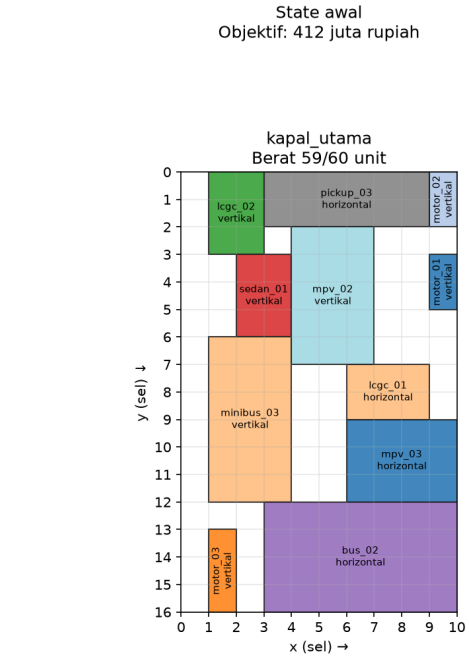


Gambar 39. Kurva objektif Simulated Annealing (run 1, seed 40)

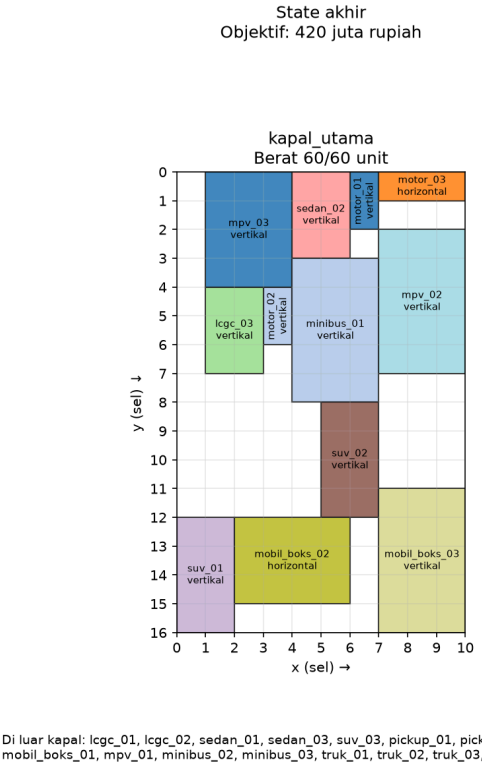


Gambar 40. Kurva e^(ΔE/T) Simulated Annealing (run 1, seed 40)

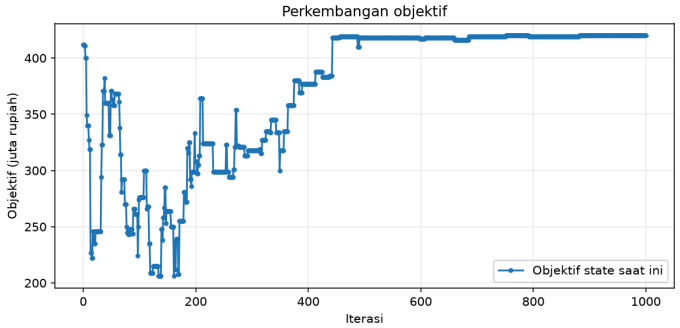
Run 2, Seed 41



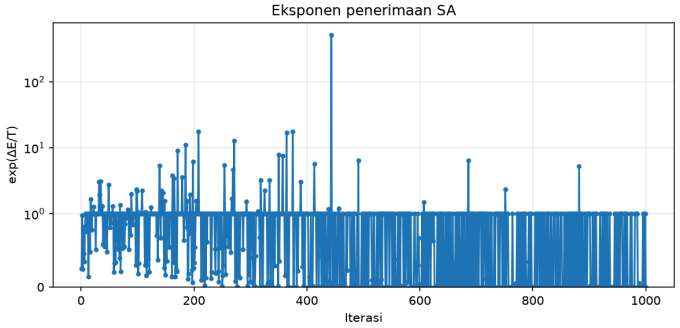
Gambar 41. State awal Simulated Annealing (run 2, seed 41)



Gambar 42. State akhir terbaik Simulated Annealing (run 2, seed 41)

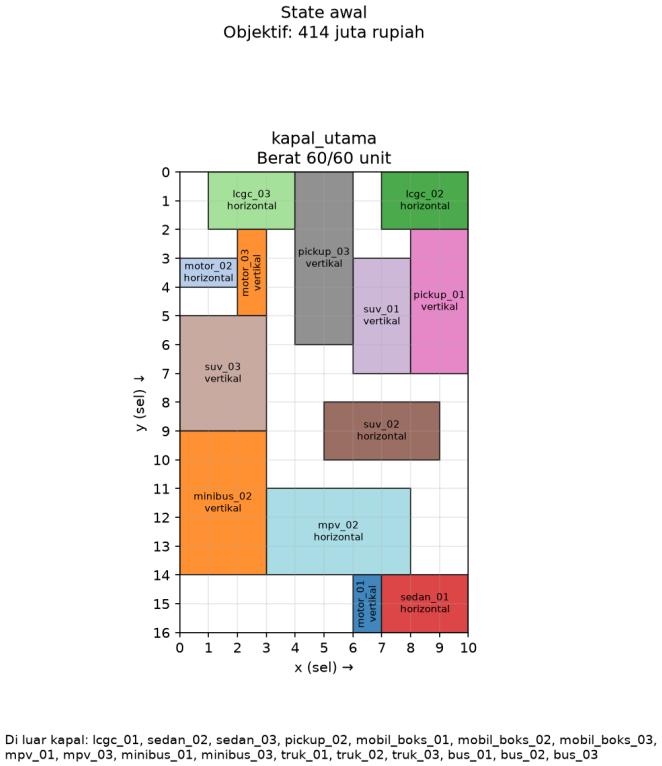


Gambar 43. Kurva objektif Simulated Annealing (run 2, seed 41)

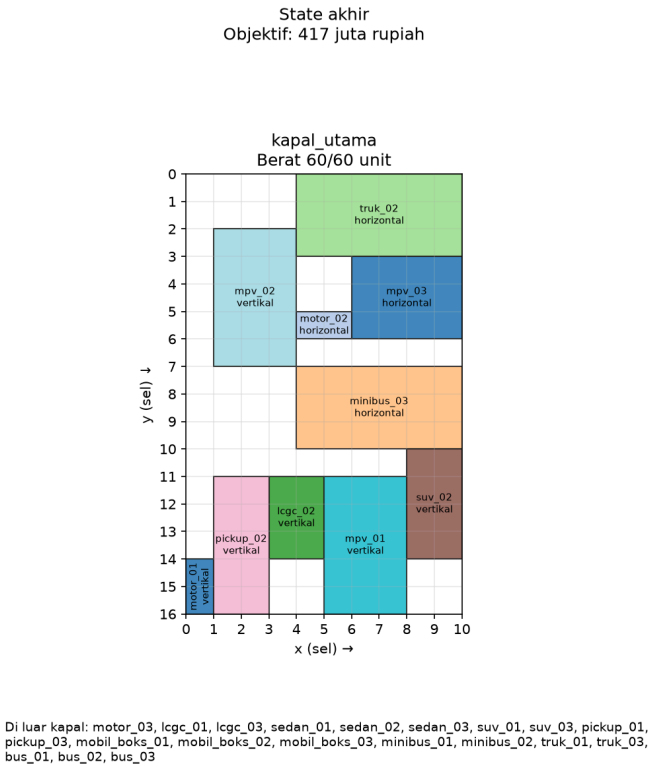


Gambar 44. Kurva e^(ΔE/T) Simulated Annealing (run 2, seed 41)

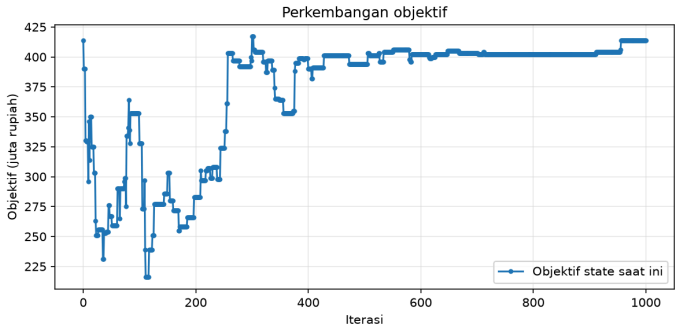
Run 3, Seed 42



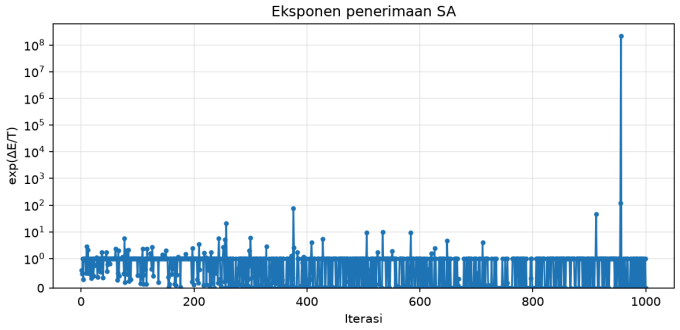
Gambar 45. State awal Simulated Annealing (run 3, seed 42)



Gambar 46. State akhir terbaik Simulated Annealing (run 3, seed 42)



Gambar 47. Kurva objektif Simulated Annealing (run 3, seed 42)



Gambar 48. Kurva e^(ΔE/T) Simulated Annealing (run 3, seed 42)

2.3.4 Eksperimen Genetic Algorithm

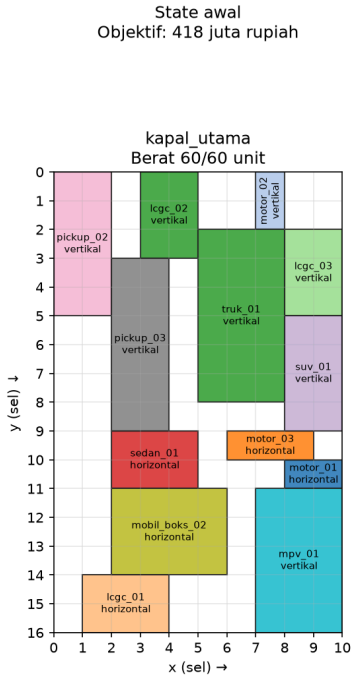
a. Jumlah populasi sebagai kontrol: 10 individu; iterasi: 1, 10, dan 100

Pada ketiga batas iterasi, objective akhir tetap 418, 414, dan 415 untuk seed 40, 41, dan 42. Penambahan iterasi belum meningkatkan individu terbaik pada pengujian ini, sedangkan durasinya bertambah.

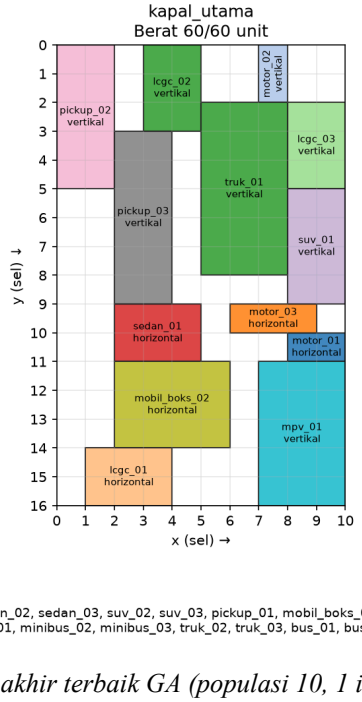
Tabel 16. Hasil GA dengan populasi 10 dan variasi jumlah iterasi

Iterasi	Run	Seed	OF Awal (terbaik)	OF Akhir (terbaik)	Durasi (s)
1	1	40	418	418	0.043718
1	2	41	414	414	0.049333
1	3	42	415	415	0.043975
10	1	40	418	418	0.252835
10	2	41	414	414	0.314496
10	3	42	415	415	0.266078
100	1	40	418	418	3.583449
100	2	41	414	414	3.612533
100	3	42	415	415	3.565832

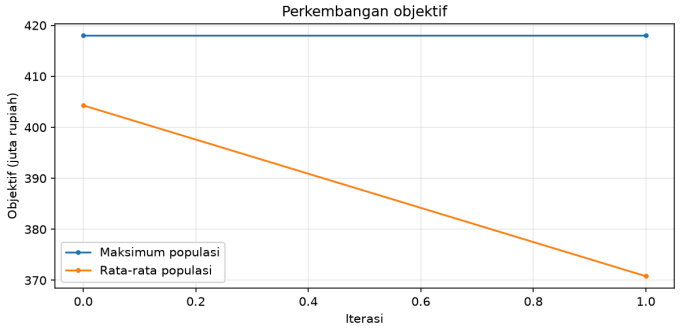
Run 1, seed 40



Gambar 49. State awal terbaik GA (populasi 10, 1 iterasi, run 1, seed 40)

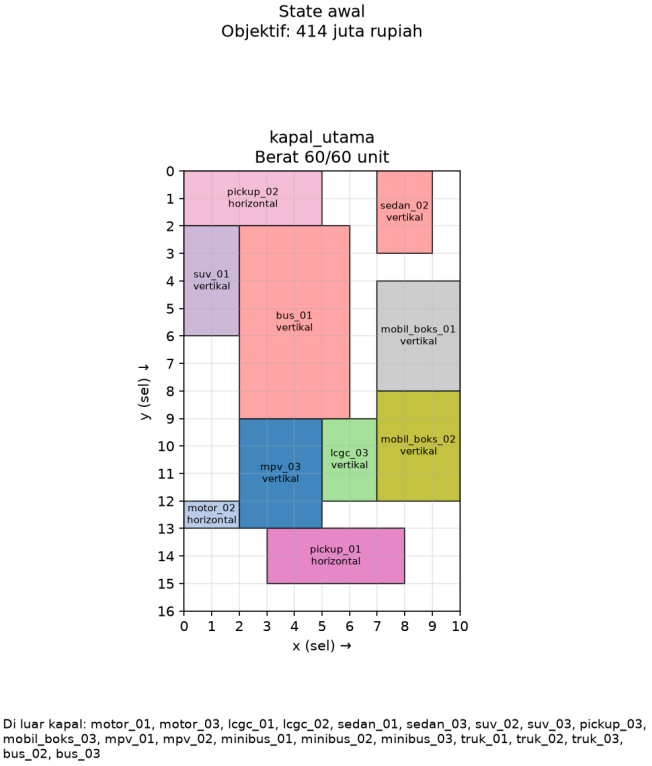


Gambar 50. State akhir terbaik GA (populasi 10, 1 iterasi, run 1, seed 40)

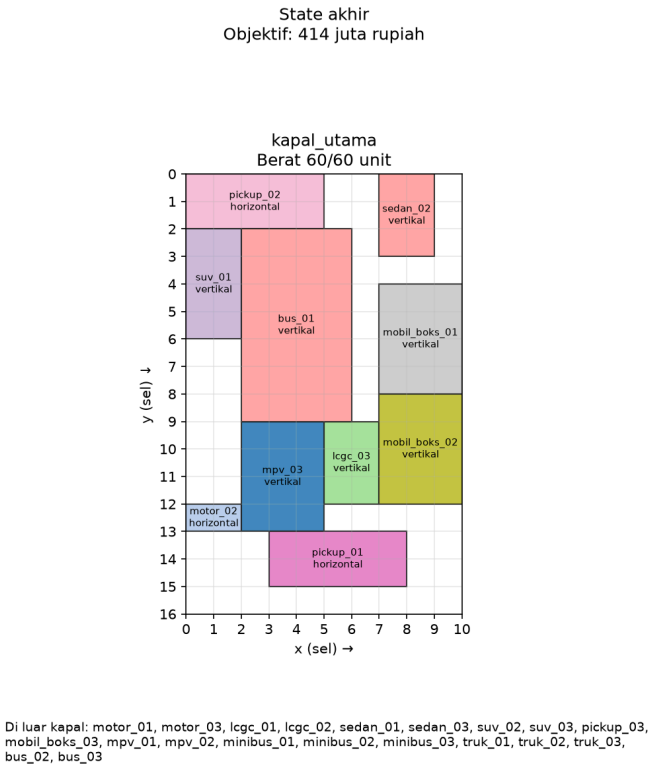


Gambar 51. Kurva objektif maksimum dan rata-rata GA (populasi 10, 1 iterasi, run 1, seed 40)

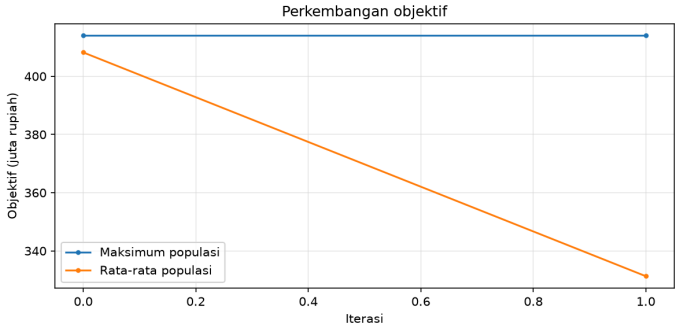
Run 2, seed 41



Gambar 52. State awal terbaik GA (populasi 10, 1 iterasi, run 2, seed 41)

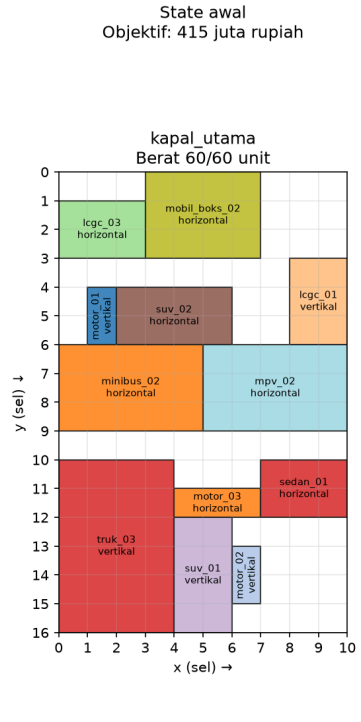


Gambar 53. State akhir terbaik GA (populasi 10, 1 iterasi, run 2, seed 41)

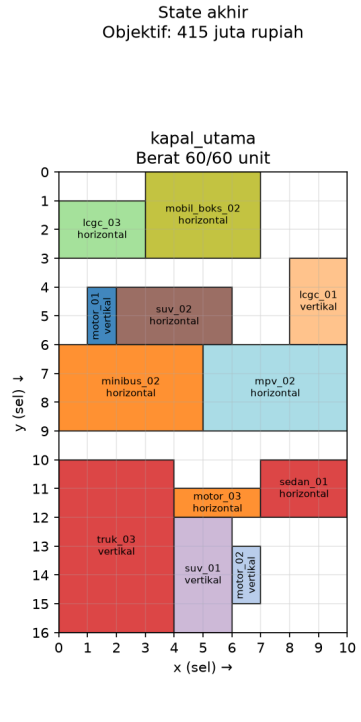


Gambar 54. Kurva objektif maksimum dan rata-rata GA (populasi 10, 1 iterasi, run 2, seed 41)

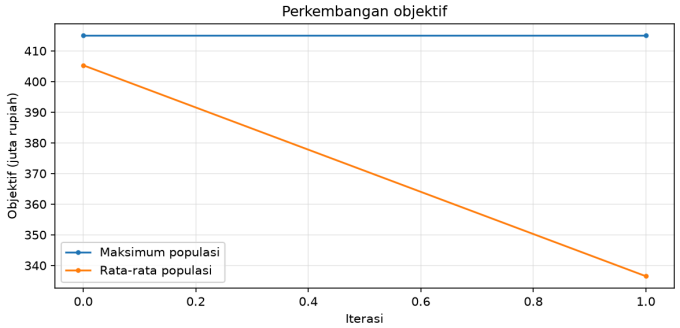
Run 3, seed 42



Gambar 55. State awal terbaik GA (populasi 10, 1 iterasi, run 3, seed 42)

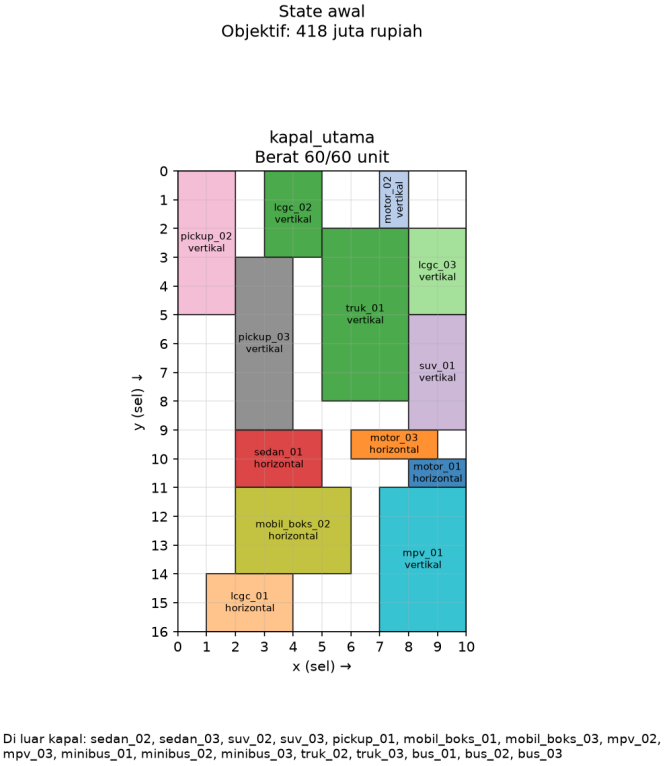


Gambar 56. State akhir terbaik GA (populasi 10, 1 iterasi, run 3, seed 42)

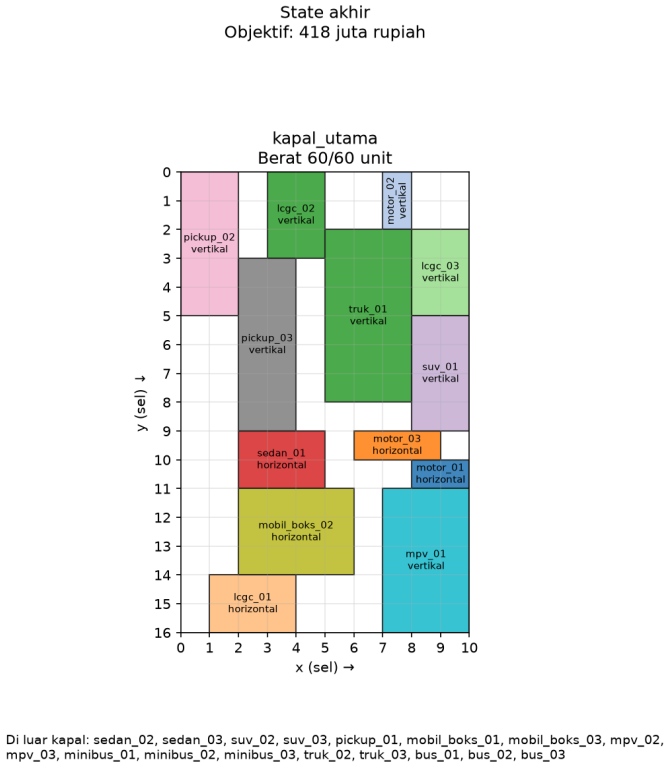


Gambar 57. Kurva objektif maksimum dan rata-rata GA (populasi 10, 1 iterasi, run 3, seed 42)

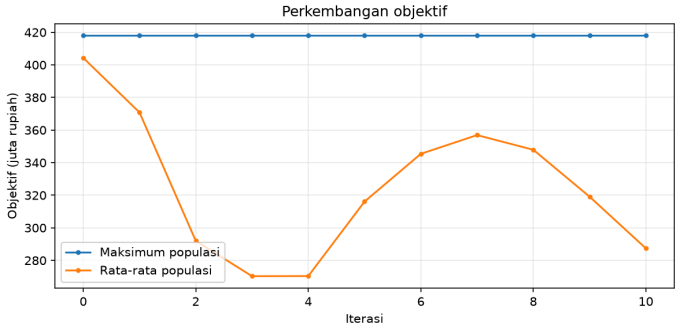
Run 1, seed 40



Gambar 58. State awal terbaik GA (populasi 10, 10 iterasi, run 1, seed 40)

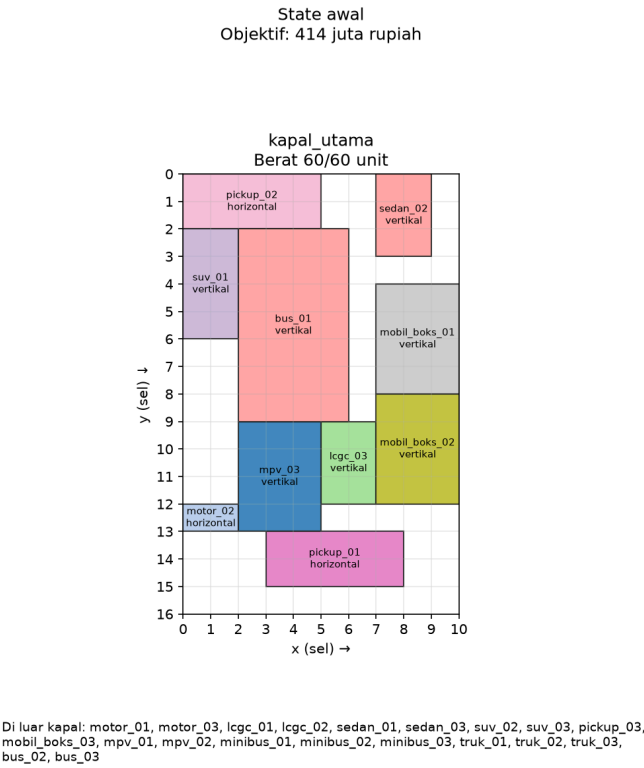


Gambar 59. State akhir terbaik GA (populasi 10, 10 iterasi, run 1, seed 40)

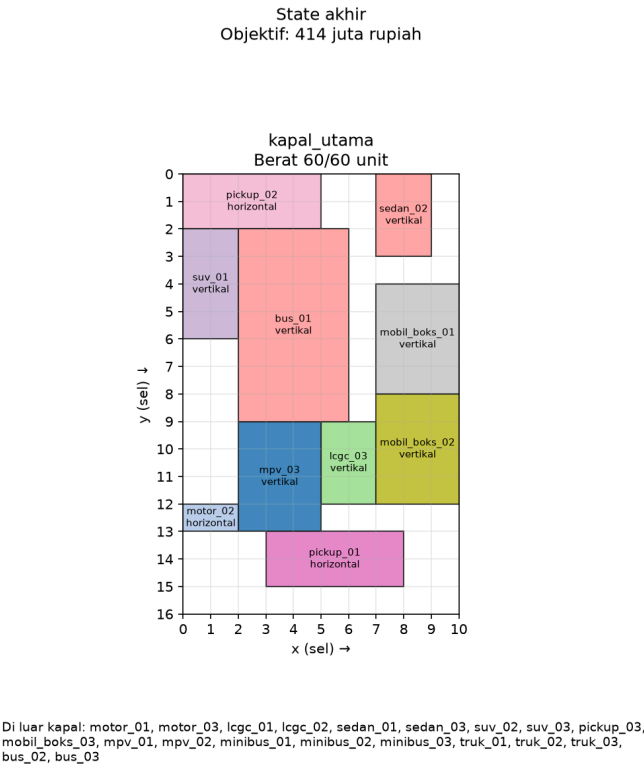


Gambar 60. Kurva objektif maksimum dan rata-rata GA (populasi 10, 10 iterasi, run 1, seed 40)

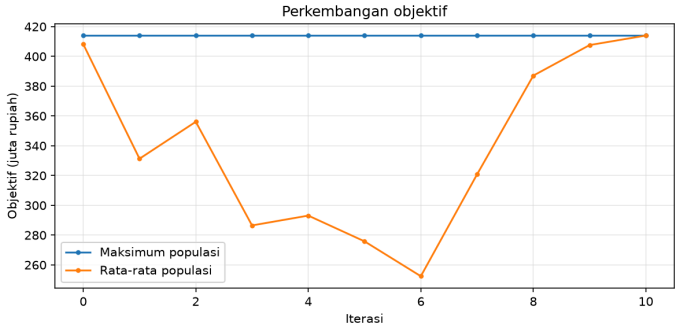
Run 2, seed 41



Gambar 61. State awal terbaik GA (populasi 10, 10 iterasi, run 2, seed 41)

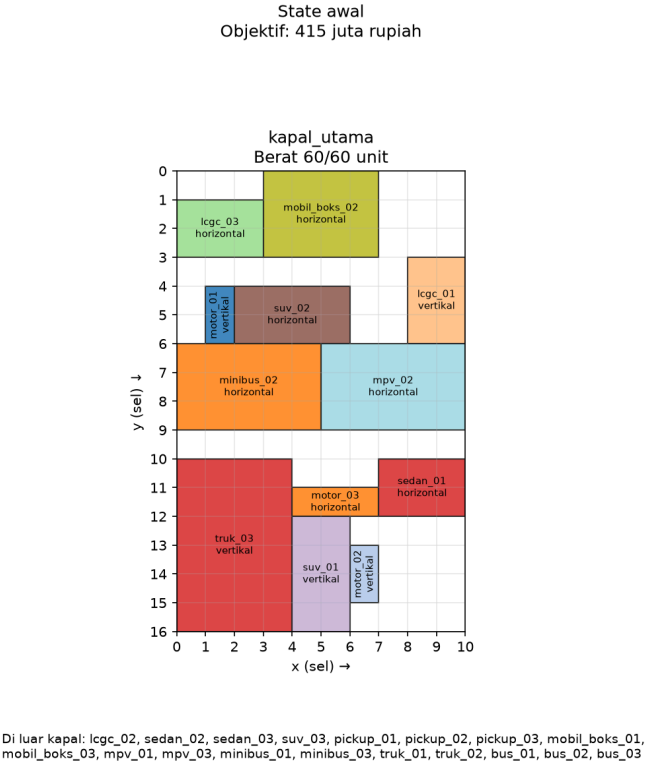


Gambar 62. State akhir terbaik GA (populasi 10, 10 iterasi, run 2, seed 41)

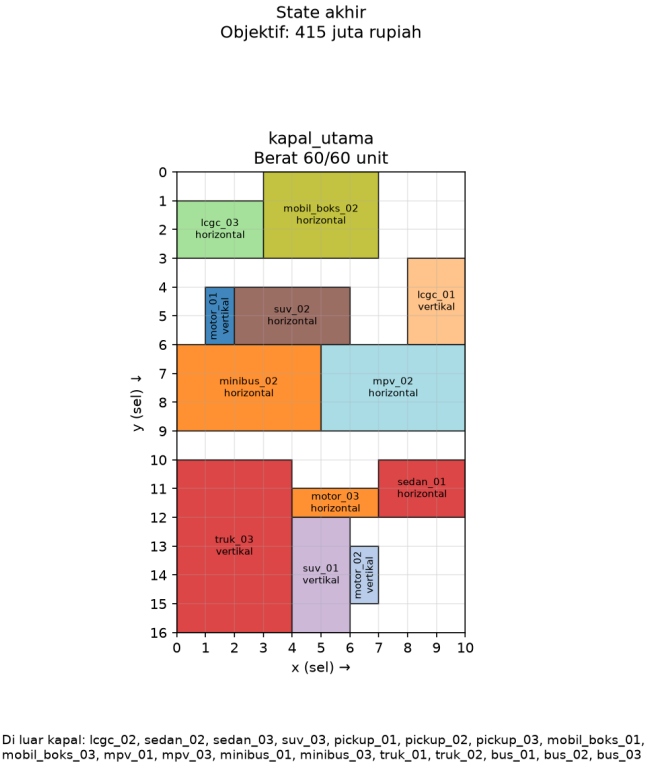


Gambar 63. Kurva objektif maksimum dan rata-rata GA (populasi 10, 10 iterasi, run 2, seed 41)

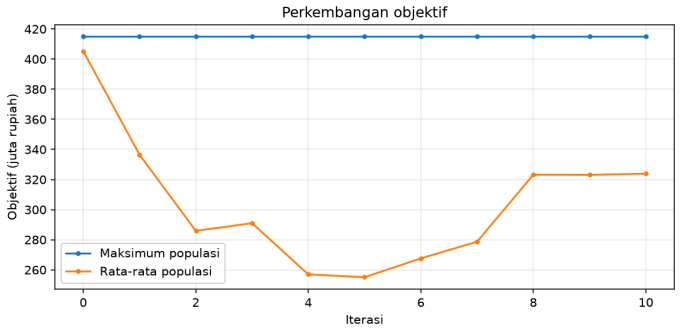
Run 3, seed 42



Gambar 64. State awal terbaik GA (populasi 10, 10 iterasi, run 3, seed 42)

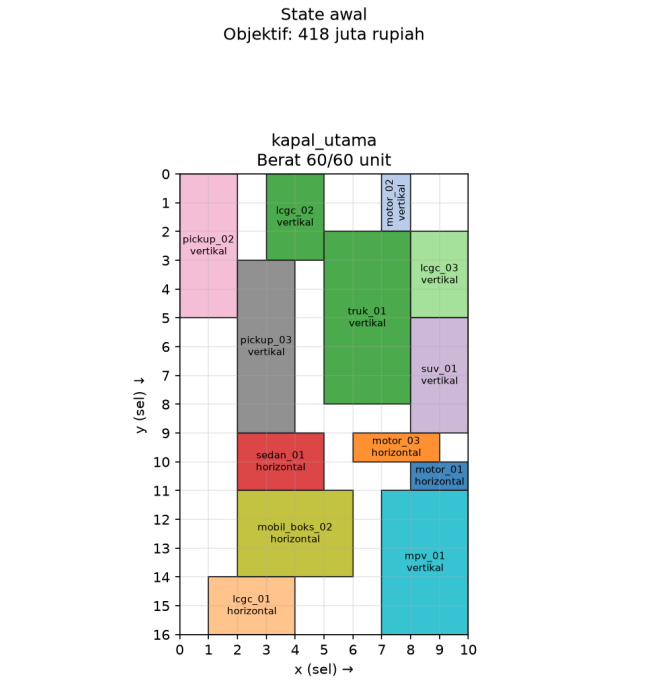


Gambar 65. State akhir terbaik GA (populasi 10, 10 iterasi, run 3, seed 42)

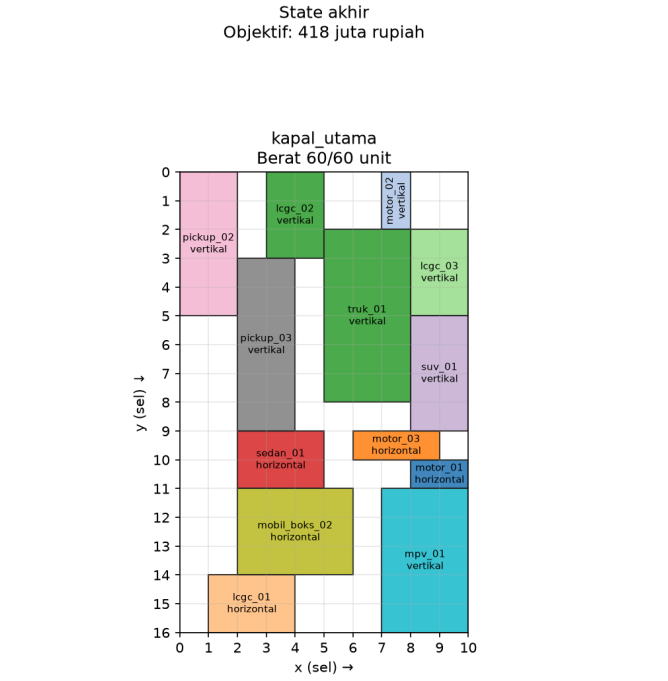


Gambar 66. Kurva objektif maksimum dan rata-rata GA (populasi 10, 10 iterasi, run 3, seed 42)

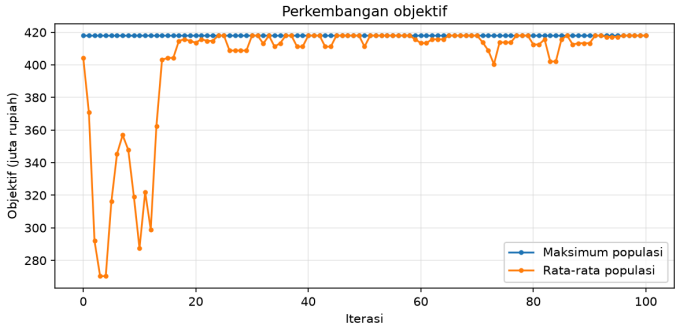
Run 1, seed 40



Gambar 67. State awal terbaik GA (populasi 10, 100 iterasi, run 1, seed 40)

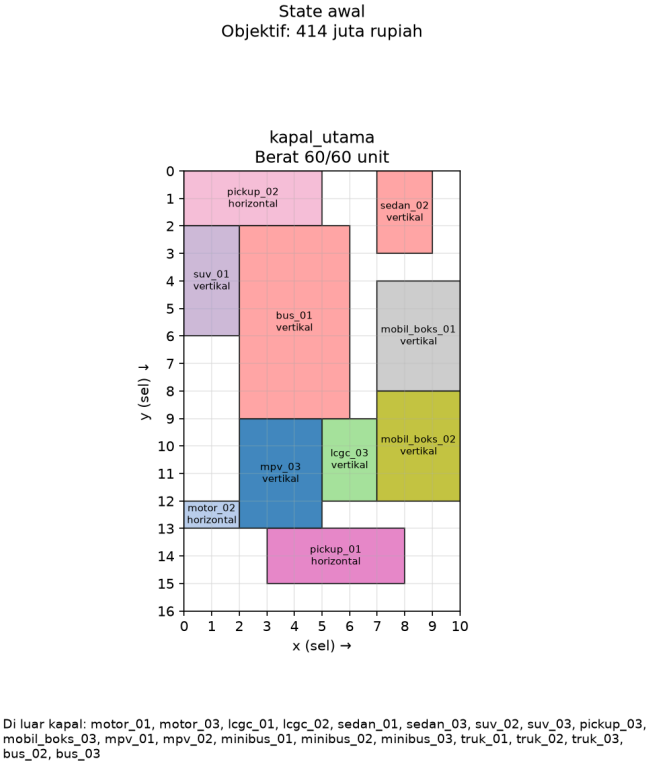


Gambar 68. State akhir terbaik GA (populasi 10, 100 iterasi, run 1, seed 40)

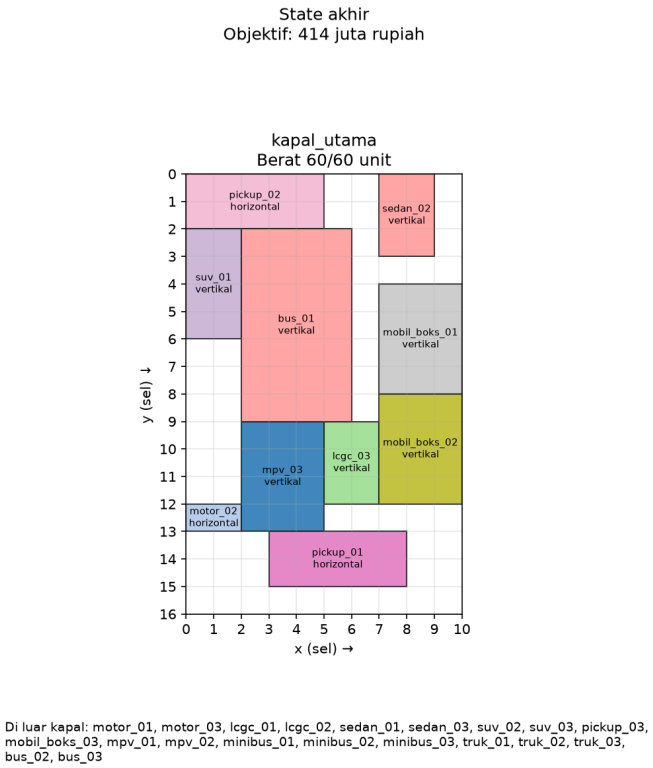


Gambar 69. Kurva objektif maksimum dan rata-rata GA (populasi 10, 100 iterasi, run 1, seed 40)

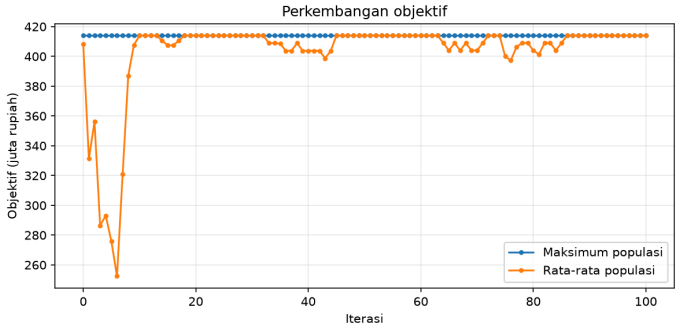
Run 2, seed 41



Gambar 70. State awal terbaik GA (populasi 10, 100 iterasi, run 2, seed 41)

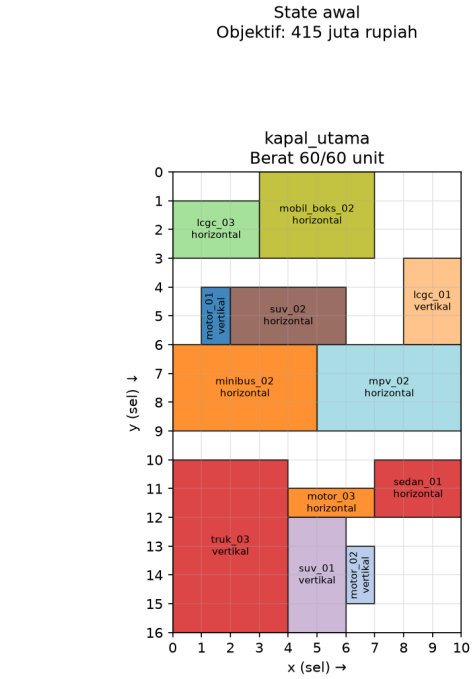


Gambar 71. State akhir terbaik GA (populasi 10, 100 iterasi, run 2, seed 41)

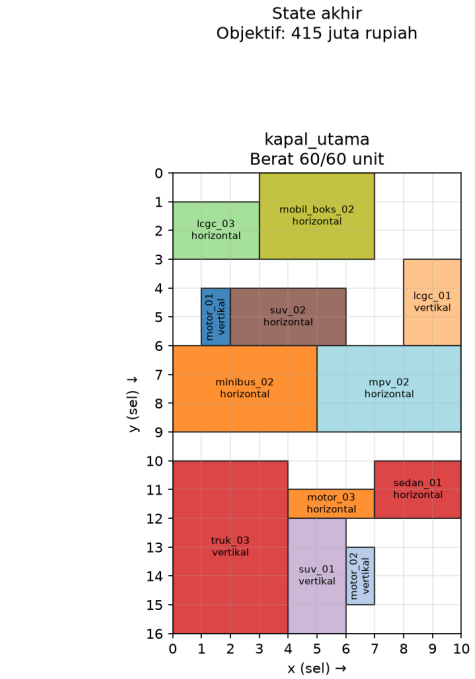


Gambar 72. Kurva objektif maksimum dan rata-rata GA (populasi 10, 100 iterasi, run 2, seed 41)

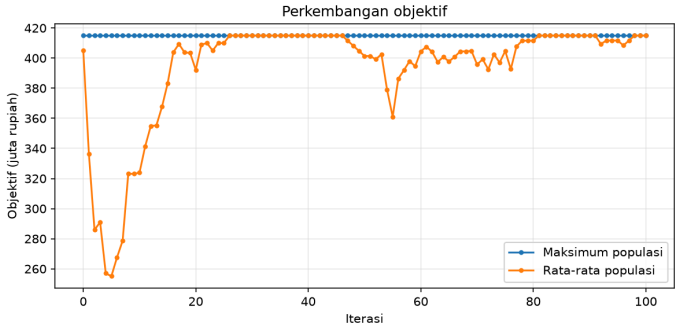
Run 3, seed 42



Gambar 73. State awal terbaik GA (populasi 10, 100 iterasi, run 3, seed 42)



Gambar 74. State akhir terbaik GA (populasi 10, 100 iterasi, run 3, seed 42)



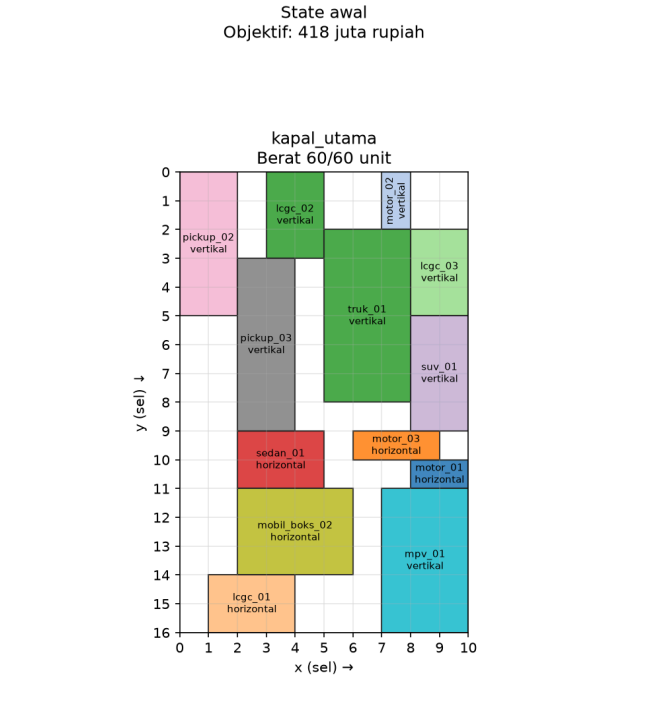
Gambar 75. Kurva objektif maksimum dan rata-rata GA (populasi 10, 100 iterasi, run 3, seed 42)

b. Iterasi sebagai kontrol: 30; populasi: 2, 10, dan 30 individu

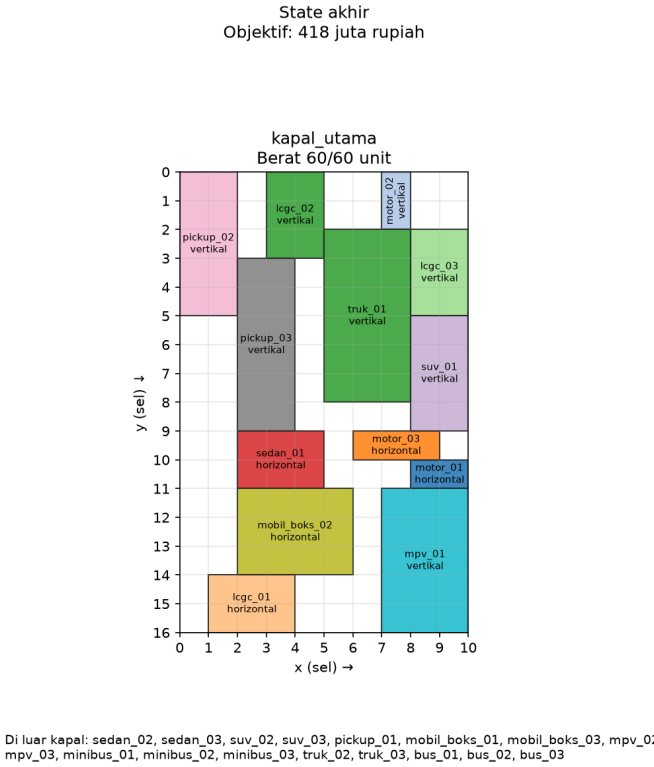
Tabel 17. Hasil GA dengan 30 iterasi dan variasi jumlah populasi

Populasi	Run	Seed	OF Awal (terbaik)	OF Akhir (terbaik)	Durasi (s)
2	1	40	418	418	0.215420
2	2	41	412	412	0.146837
2	3	42	415	415	0.193706
10	1	40	418	418	1.022651
10	2	41	414	414	1.044337
10	3	42	415	415	0.956969
30	1	40	422	425	2.604693
30	2	41	422	422	2.716269
30	3	42	419	419	2.539227

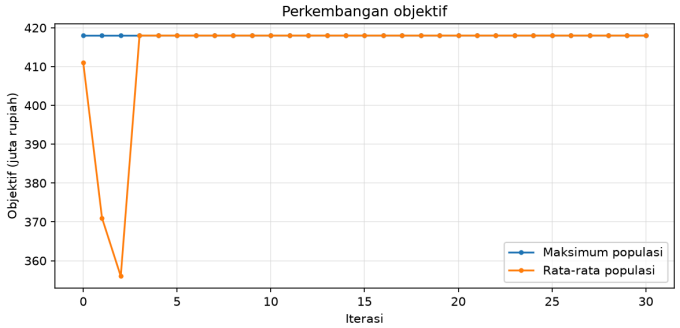
Run 1, seed 40



Gambar 76. State awal terbaik GA (populasi 2, 30 iterasi, run 1, seed 40)

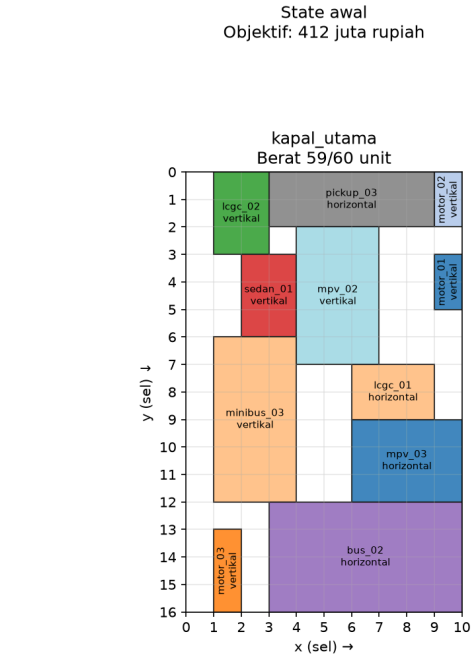


Gambar 77. State akhir terbaik GA (populasi 2, 30 iterasi, run 1, seed 40)



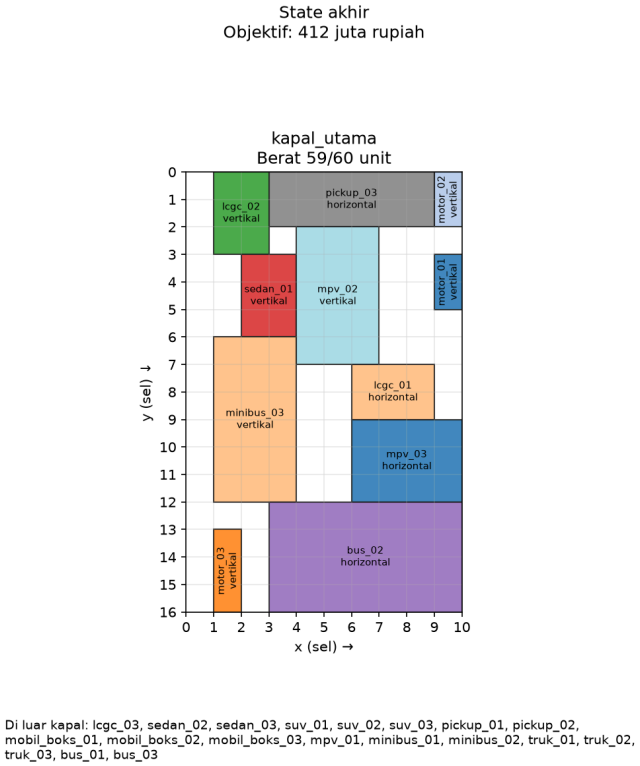
Gambar 78. Kurva objektif maksimum dan rata-rata GA (populasi 2, 30 iterasi, run 1, seed 40)

Run 2, seed 41

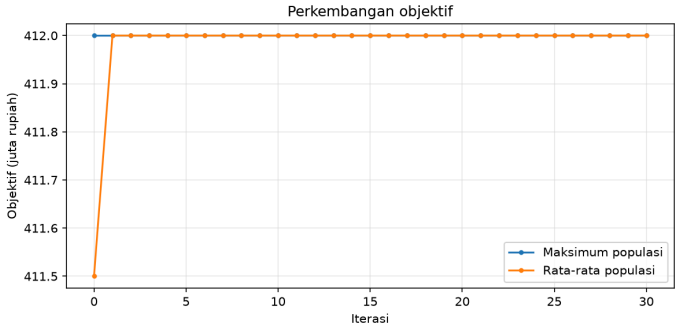


Di luar kapal: lcgc_03, sedan_02, sedan_03, suv_01, suv_02, suv_03, pickup_01, pickup_02, mobil_boks_01, mobil_boks_02, mobil_boks_03, mpv_01, minibus_01, minibus_02, truk_01, truk_02, truk_03, bus_01, bus_03

Gambar 79. State awal terbaik GA (populasi 2, 30 iterasi, run 2, seed 41)

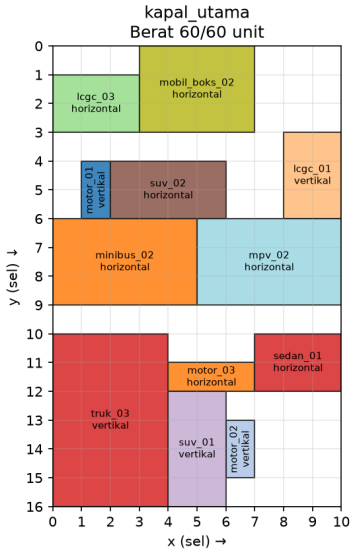


Gambar 80. State akhir terbaik GA (populasi 2, 30 iterasi, run 2, seed 41)

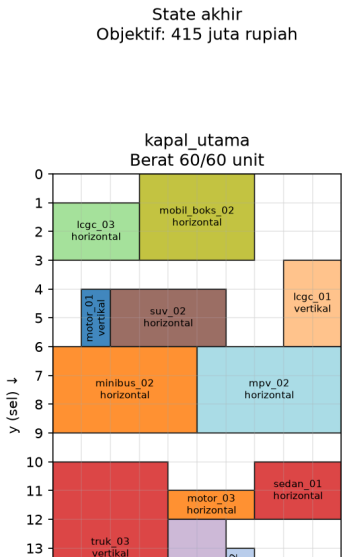


Gambar 81. Kurva objektif maksimum dan rata-rata GA (populasi 2, 30 iterasi, run 2, seed 41)

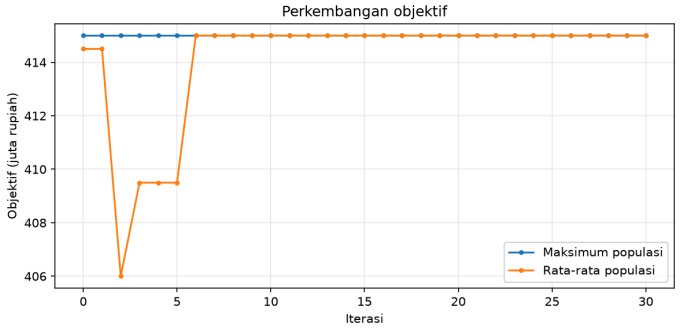
Run 3, seed 42



Gambar 82. State awal terbaik GA (populasi 2, 30 iterasi, run 3, seed 42)

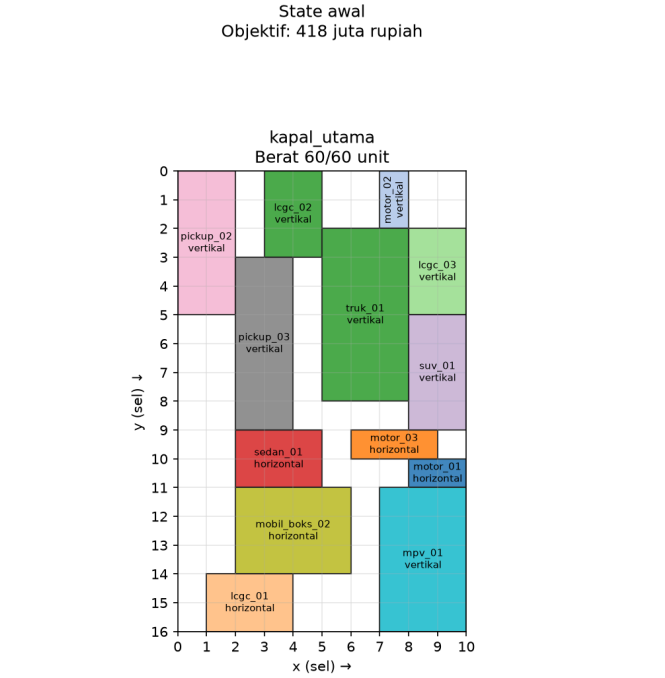


Gambar 83. State akhir terbaik GA (populasi 2, 30 iterasi, run 3, seed 42)



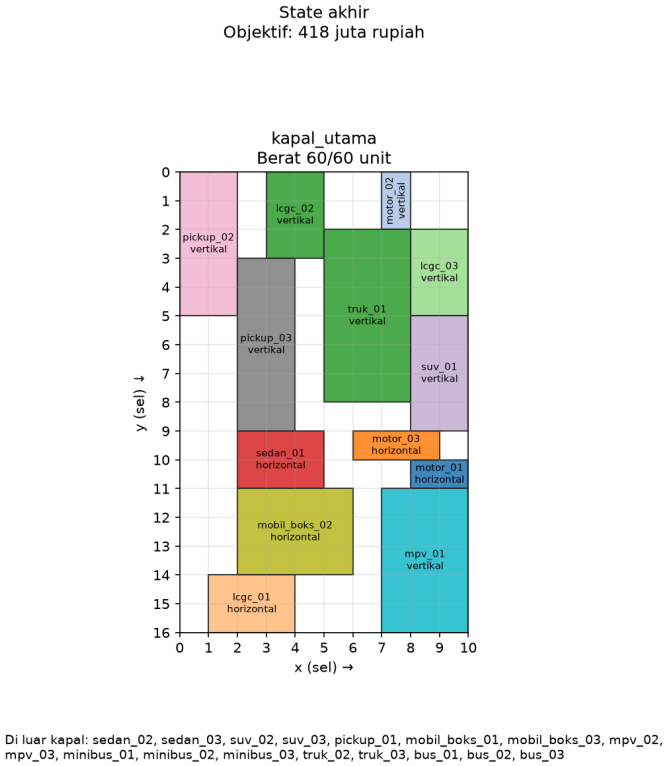
Gambar 84. Kurva objektif maksimum dan rata-rata GA (populasi 2, 30 iterasi, run 3, seed 42)

Run 1, seed 40

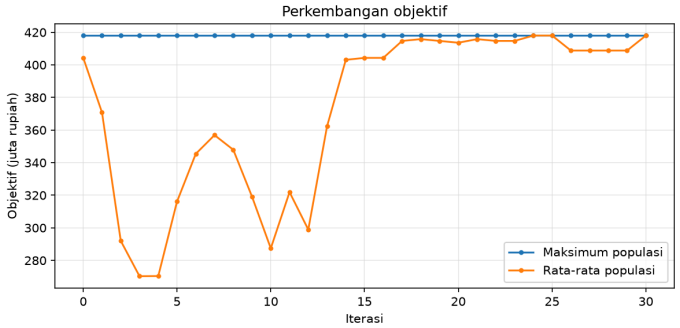


Di luar kapal: sedan_02, sedan_03, suv_02, suv_03, pickup_01, mobil_boks_01, mobil_boks_03, mpv_02, mpv_03, minibus_01, minibus_02, minibus_03, truk_02, truk_03, bus_01, bus_02, bus_03

Gambar 85. State awal terbaik GA (populasi 10, 30 iterasi, run 1, seed 40)

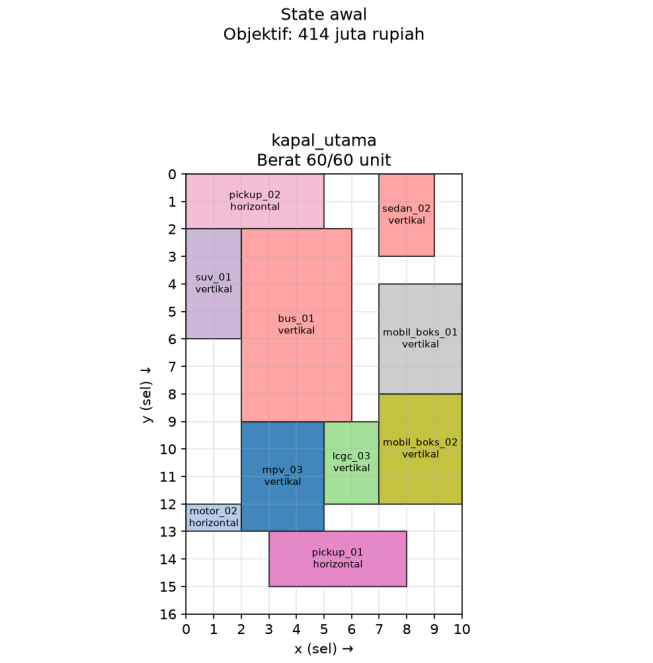


Gambar 86. State akhir terbaik GA (populasi 10, 30 iterasi, run 1, seed 40)

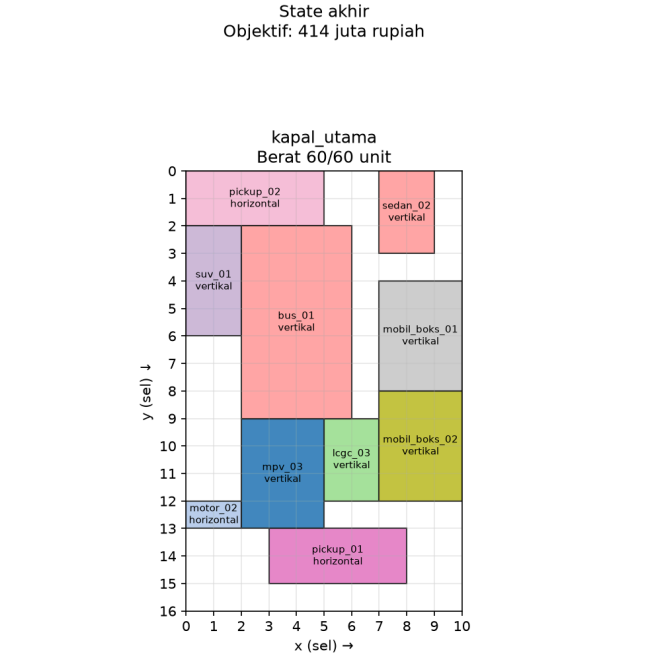


Gambar 87. Kurva objektif maksimum dan rata-rata GA (populasi 10, 30 iterasi, run 1, seed 40)

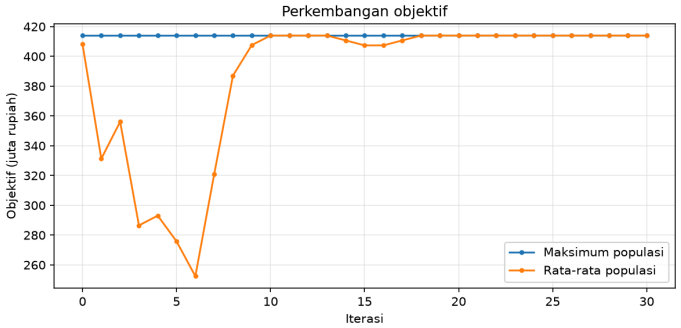
Run 2, seed 41



Gambar 88. State awal terbaik GA (populasi 10, 30 iterasi, run 2, seed 41)

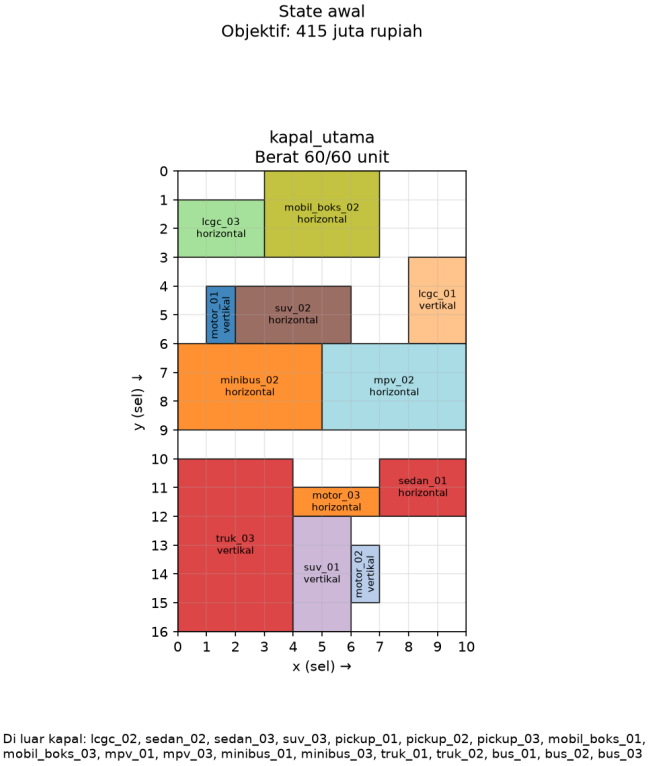


Gambar 89. State akhir terbaik GA (populasi 10, 30 iterasi, run 2, seed 41)

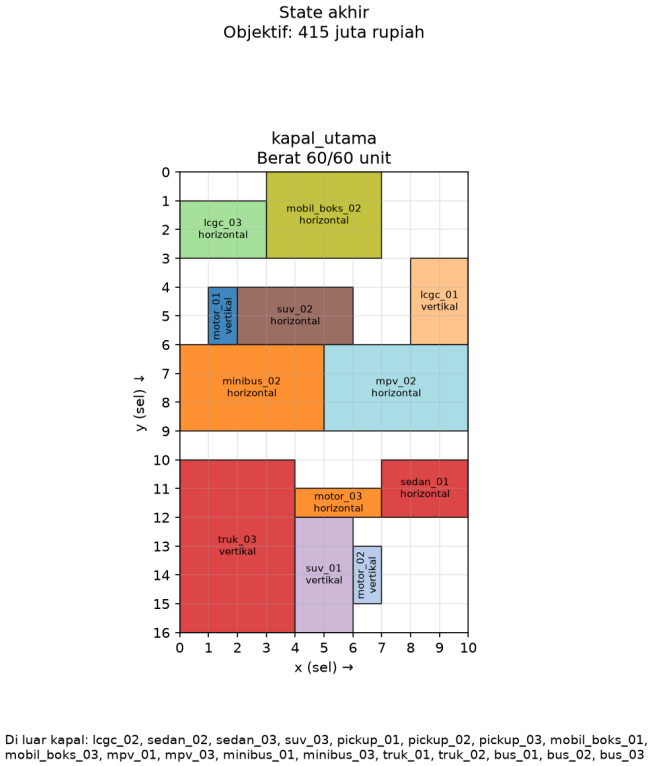


Gambar 90. Kurva objektif maksimum dan rata-rata GA (populasi 10, 30 iterasi, run 2, seed 41)

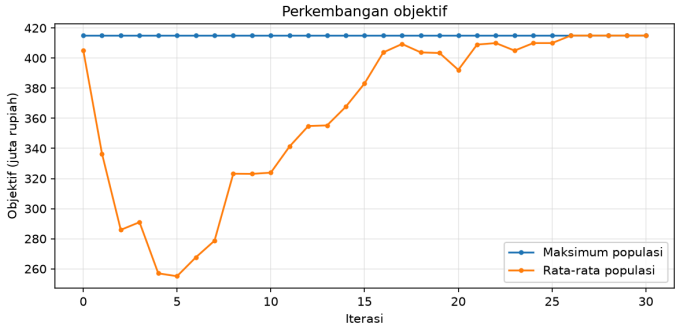
Run 3, seed 42



Gambar 91. State awal terbaik GA (populasi 10, 30 iterasi, run 3, seed 42)

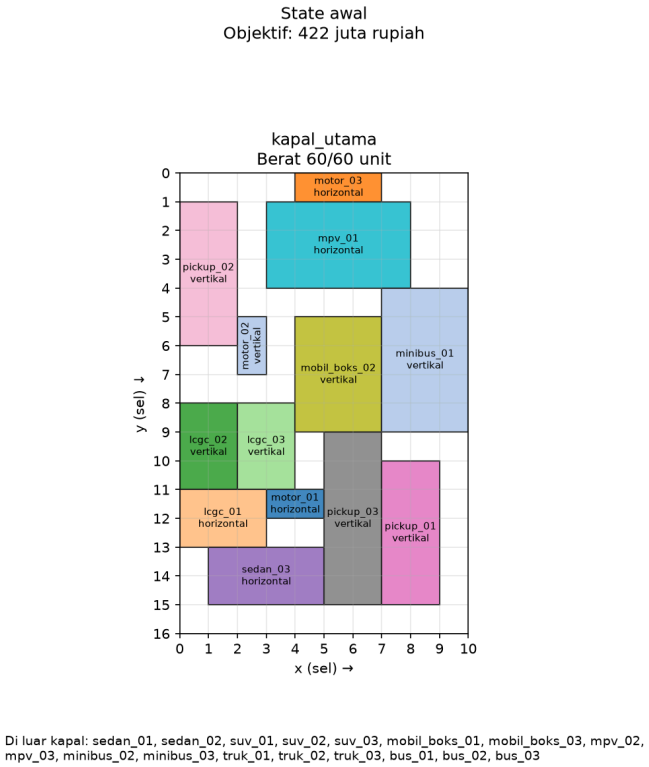


Gambar 92. State akhir terbaik GA (populasi 10, 30 iterasi, run 3, seed 42)

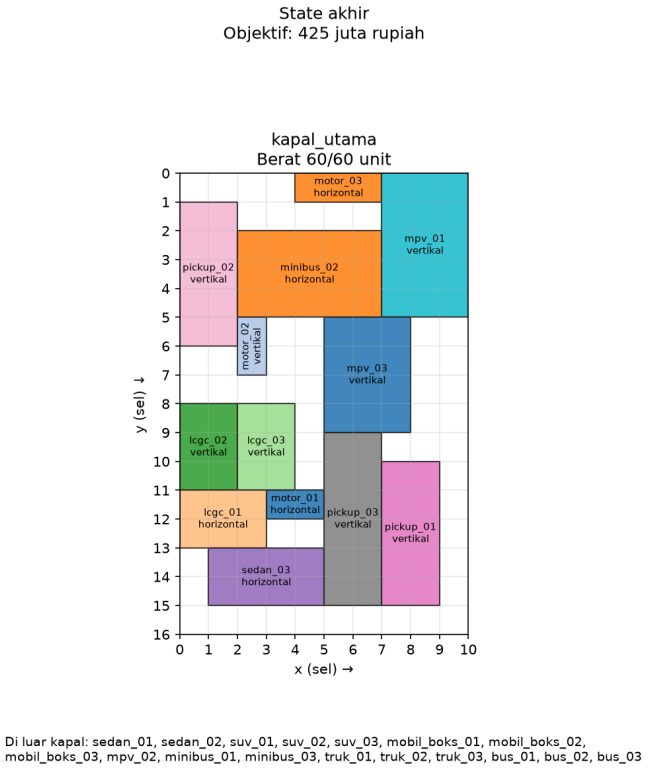


Gambar 93. Kurva objektif maksimum dan rata-rata GA (populasi 10, 30 iterasi, run 3, seed 42)

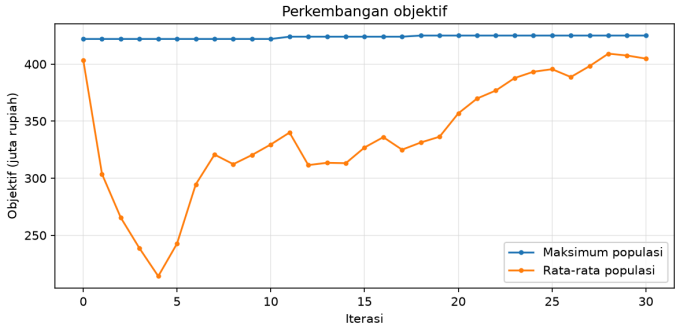
Run 1, seed 40



Gambar 94. State awal terbaik GA (populasi 30, 30 iterasi, run 1, seed 40)

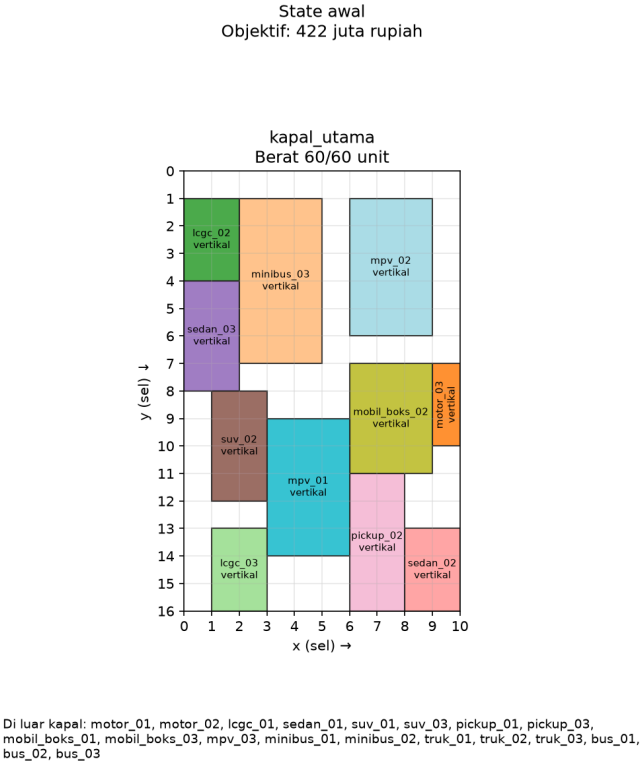


Gambar 95. State akhir terbaik GA (populasi 30, 30 iterasi, run 1, seed 40)



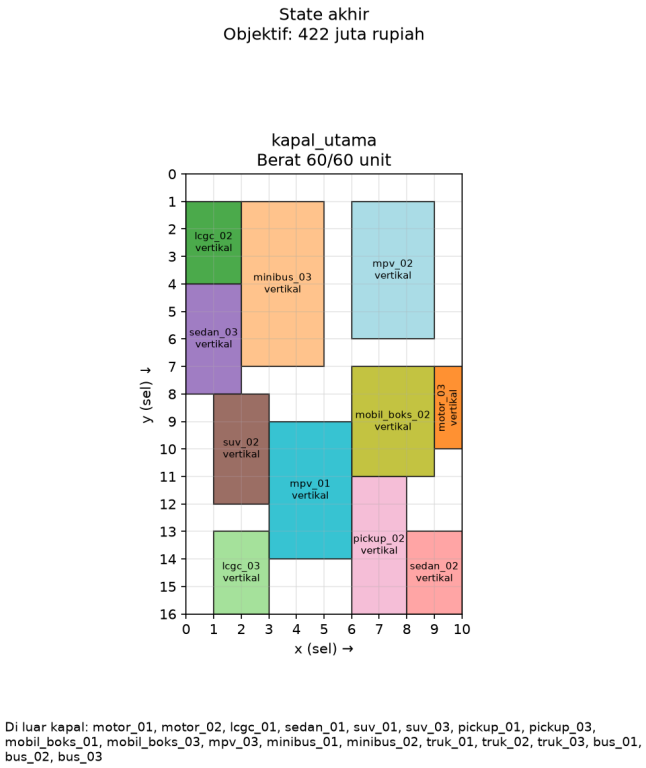
Gambar 96. Kurva objektif maksimum dan rata-rata GA (populasi 30, 30 iterasi, run 1, seed 40)

Run 2, seed 41

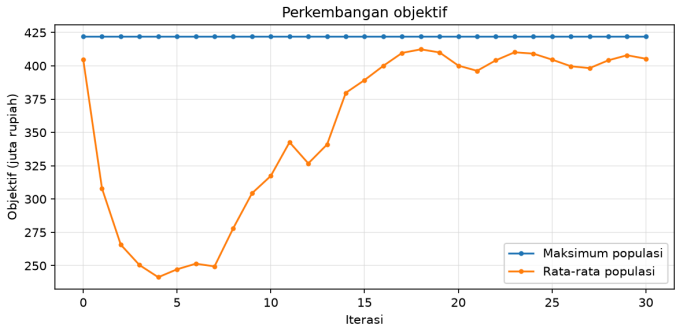


Di luar kapal: motor_01, motor_02, lcgc_01, sedan_01, suv_01, suv_03, pickup_01, pickup_03, mobil_boks_01, mobil_boks_03, mpv_03, minibus_01, minibus_02, truk_01, truk_02, truk_03, bus_01, bus_02, bus_03

Gambar 97. State awal terbaik GA (populasi 30, 30 iterasi, run 2, seed 41)

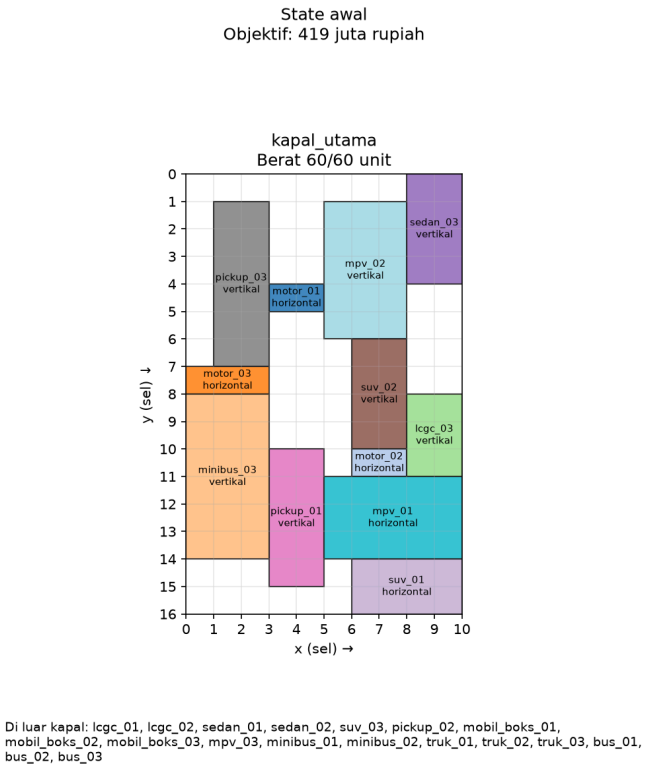


Gambar 98. State akhir terbaik GA (populasi 30, 30 iterasi, run 2, seed 41)

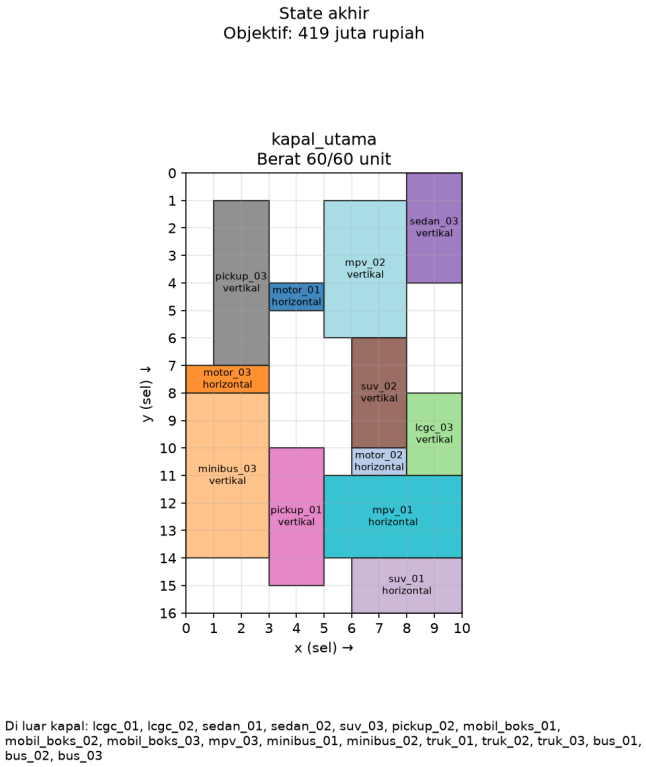


Gambar 99. Kurva objektif maksimum dan rata-rata GA (populasi 30, 30 iterasi, run 2, seed 41)

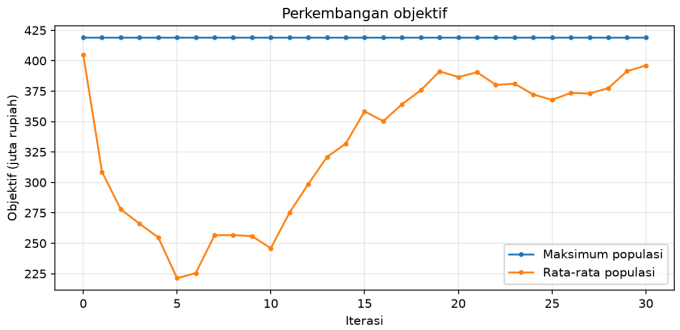
Run 3, seed 42



Gambar 100. State awal terbaik GA (populasi 30, 30 iterasi, run 3, seed 42)



Gambar 101. State akhir terbaik GA (populasi 30, 30 iterasi, run 3, seed 42)



Gambar 102. Kurva objektif maksimum dan rata-rata GA (populasi 30, 30 iterasi, run 3, seed 42)

2.4 Analisis Hasil Eksperimen

2.4.1 Kedekatan dengan Global Optima

Batas atas objective dihitung menggunakan 0/1 knapsack dengan kapasitas 60 unit dan mengabaikan batas ruang serta penempatan. Relaksasi ini menghasilkan 435 juta rupiah. Karena batasan yang diabaikan hanya memperluas pilihan solusi, setiap solusi penempatan valid memiliki nilai paling tinggi 435.

Solusi terbaik teramati adalah 429 juta rupiah pada random restart seed 40, atau 98.62% dari batas atas. Dengan demikian, optimum global berada pada rentang 429 sampai 435 juta rupiah dan selisih solusi terbaik terhadap optimum paling banyak 6 juta rupiah. Pencapaian optimum global belum terbukti.

2.4.2 Perbandingan Hasil Antar Algoritma

Tabel 18. Perbandingan hasil dan durasi seluruh konfigurasi

Konfigurasi	Rata-Rata OF Akhir	Minimum	Maksimum	SD Sampel	Rata-Rata durasi (s)
HC stochastic	418.667	410	425	7.767	0.882485
Simulated Annealing	416.667	413	420	3.512	0.546154
GA P=10, I=1	415.667	414	418	2.082	0.045675
GA P=10, I=10	415.667	414	418	2.082	0.277803

GA P=10, I=100	415.667	414	418	2.082	3.587271
GA P=2, I=30	415.000	412	418	3.000	0.185321
GA P=10, I=30	415.667	414	418	2.082	1.007986
GA P=30, I=30	422.000	419	425	3.000	2.620063
HC steepest	418.333	409	425	8.327	0.071201
HC sideways	419.333	409	425	8.963	1.594323
HC random restart	426.000	423	429	3.000	0.828751

Pada bagian wajib, GA populasi 30 dan iterasi 30 memiliki rata-rata tertinggi, yaitu 422.000, diikuti HC stochastic 418.667 dan SA 416.667 juta rupiah. Jika bonus disertakan, random restart tertinggi dengan rata-rata 426.000. Perbandingan ini berlaku untuk konfigurasi dan tiga seed yang diuji.

GA memulai pencarian dari individu terbaik suatu populasi sehingga objective awal dapat lebih tinggi daripada HC dan SA. Kenaikan rata-rata GA populasi 30 hanya 1 juta rupiah dari populasi awal, sedangkan HC stochastic naik 9.333 dan SA naik 6.667 juta rupiah. Nilai akhir GA yang tinggi perlu dibaca bersama kualitas populasi awalnya.

2.4.3 Perbandingan Durasi Pencarian

Di antara varian HC yang diuji, steepest paling cepat dengan rata-rata 0.071201 detik dan tiga langkah per run. Random restart memerlukan 0.828751 detik, sedangkan sideways 1.594323 detik karena menjalankan 100 langkah sideways. Durasi stochastic sebesar 0.882485 detik dipengaruhi batas 1.000 kegagalan beruntun.

SA memerlukan rata-rata 0.546154 detik untuk 1.000 iterasi. GA populasi 10 meningkat dari 0.045675 detik pada satu iterasi menjadi 3.587271 detik pada 100 iterasi. Pada 30 iterasi, kenaikan populasi dari 2 menjadi 30 meningkatkan durasi dari 0.185321 menjadi 2.620063

detik. Durasi ini berasal dari satu pengukuran pencarian per run dan tidak termasuk pembuatan gambar.

Sideways memiliki simpangan baku terbesar, yaitu 8.963 juta rupiah, dengan hasil 409, 424, dan 425. SA memiliki simpangan baku 3.512. Frekuensi *stuck* yang rendah tidak otomatis menghasilkan objective tertinggi. Tiga pengulangan memberikan gambaran awal dan belum mewakili seluruh kemungkinan state awal.

2.4.4 Konsistensi Hasil

GA populasi 10 menghasilkan simpangan baku 2.082 juta rupiah pada seluruh batas iterasi yang diuji. Hasilnya konsisten karena individu terbaik belum berubah. GA populasi 30 dan random restart sama-sama memiliki simpangan baku 3.000, dengan rata-rata masing-masing 422.000 dan 426.000.

2.4.5 Pengaruh Iterasi dan Populasi pada Genetic Algorithm

Penambahan iterasi dari 1 menjadi 10 dan 100 pada populasi 10 tidak meningkatkan objective maksimum: hasil tetap 418, 414, dan 415. Elitisme mempertahankan individu terbaik, sementara perubahan rata-rata populasi pada grafik menunjukkan bahwa proses pembentukan anak tetap berjalan. Tambahan iterasi pada konfigurasi ini terutama menambah durasi.

Pada 30 iterasi, populasi 30 menghasilkan rata-rata 422.000, lebih tinggi daripada populasi 2 sebesar 415.000 dan populasi 10 sebesar 415.667. Sebagian besar selisih sudah ada pada populasi awal. Perbaikan selama pencarian terjadi pada seed 40, dari 422 menjadi 425. Karena itu, keuntungan populasi lebih besar pada pengujian ini terutama berasal dari cakupan individu awal yang lebih luas, dengan biaya evaluasi lebih besar.

BAB III KESIMPULAN DAN SARAN

3.1 Kesimpulan

HC, SA, dan GA berhasil digunakan untuk mencari penempatan kendaraan yang memenuhi batas dek, kapasitas, dan aturan tidak bertumpuk. Dari 33 run, hasil terbaik diperoleh random restart sebesar 429 juta rupiah, dengan rata-rata 426 juta rupiah. Untuk algoritma wajib, konfigurasi GA populasi 30 dan 30 iterasi memiliki rata-rata tertinggi sebesar 422 juta rupiah, diikuti stochastic HC sebesar 418,667 dan SA sebesar 416,667 juta rupiah.

Penambahan iterasi GA dari 1 menjadi 100 pada populasi 10 belum meningkatkan objective terbaik, tetapi menambah durasi. Populasi 30 memberikan hasil lebih tinggi, dengan sebagian besar keunggulan sudah berasal dari populasi awal. Hasil terbaik 429 mencapai 98,62% dari batas atas 435, tetapi optimum global belum dibuktikan. Kesimpulan ini berlaku pada dataset, parameter, dan tiga seed yang diuji.

3.2 Saran

Pengujian selanjutnya dapat menggunakan lebih banyak seed serta variasi dataset, ukuran dek, dan kapasitas untuk menilai konsistensi hasil. Pembanding optimum yang mempertimbangkan penempatan kendaraan juga diperlukan untuk memperjelas jarak solusi terhadap optimum global.

Pada GA, peluang mutasi, crossover, dan mekanisme perbaikan anak dapat dievaluasi untuk mengurangi stagnasi. Parameter suhu dan pendinginan SA juga dapat diuji lebih lanjut. Setiap perubahan sebaiknya dievaluasi dengan parameter lain tetap agar pengaruhnya dapat diamati.

PEMBAGIAN TUGAS

Tabel 19. Pembagian tugas anggota kelompok

NIM	Nama	Tugas
18224005	Mishael Gilland	Inisiasi Repo, membuat seluruh algoritma hill climbing
18224067	Bryant Azraqi Mohammad	Membuat Algoritma Genetic, membuat state, representasi state, dan dataset, mengatur logika testing akhir.
18224117	Stephanie Mae	Membuat kode visualisasi, membuat kode algoritma simulated annealing

REFERENSI

[1] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=oGTa9kGE3P8. [Accessed: Sep. 16, 2026].

[2] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Sep. 16, 2026].

[3] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Sep. 16, 2026].

[4] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=AdhG7HnhYrk. [Accessed: Sep. 16, 2026].

[5] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=aI41TGo9J9c. [Accessed: Sep. 16, 2026].

[6] Matplotlib Development Team, “Pyplot tutorial,” *Matplotlib Documentation*. [Online]. Available: https://matplotlib.org/stable/tutorials/pyplot.html. [Accessed: Oct. 18, 2026].

[7] Matplotlib Development Team, “matplotlib.patches.Rectangle,” *Matplotlib Documentation*. [Online]. Available: https://matplotlib.org/stable/api/_as_gen/matplotlib.patches.Rectangle.html. [Accessed: Oct. 18, 2026].

[8] Python Software Foundation, “random: Generate pseudo-random numbers,” *Python 3 Documentation*. [Online]. Available: https://docs.python.org/3/library/random.html. [Accessed: Oct. 18, 2026].

[9] Python Software Foundation, “time: Time access and conversions,” *Python 3 Documentation*. [Online]. Available: https://docs.python.org/3/library/time.html#time.perf_counter. [Accessed: Oct. 18, 2026].

[10] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

LAMPIRAN

Lampiran A. Form Penggunaan Generative AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, ~~saya~~/kami*:

Nama/NIM : Mishael Gilland/18224005, Bryant Azraqi M/18224067, Stephanie Mae/18224117

Fakultas/Sekolah : STEI-K

Kode Mata Kuliah : IF-3070

Nama Mata Kuliah : Dasar Intelegensi Artifisial

Judul Tugas/Proposal :Implementasi Algoritma Local Search untuk Optimasi Penempatan Kendaraan pada Geladak Kapal Kargo

Menyatakan bahwa ~~saya~~/kami* menggunakan/~~tidak~~ ~~menggunakan~~* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Tabel 20. Deklarasi penggunaan AI

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	~~Ya~~/Tidak*	No AI
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/~~Tidak~~*	AI-assisted idea generation and structuring
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/~~Tidak~~*	AI task completion-Humanevaluation
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	~~Ya~~/Tidak*	No AI

~~Saya~~/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Bryant Azraqi Mohammad

18224067

Lampiran B. Tautan Repository

https://github.com/Soyuzn/IF3070_G34_TIRTA